A Major Project Report

on

COGNIFY AI Driven Learning, Visualization & Gamification

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

CSE (DATA SCIENCE)

By

A Rushmitha (22AG1A6770)
A Shivathmika (22AG1A6765)
R Sindhu Meghana (22AG1A67B5)
D Varun Kumar (22AG1A6787)

Under the guidance of

Mrs. P Niharika
Assistant Professor

DEPARTMENT OFCSE(DATA SCIENCE)

ACE Engineering College

Ankushapur(V), Ghatkesar(M), MedchalDist - 501 301

(An Autonomous Institution, Affiliated to JNTUH, Hyderabad)

www.aceec.ac.in

A.Y:2025-2026

ACE
Engineering College
UGC AUTONOMOUS INSTITUTION
(Sponsored by Yadala Satyanarayana Memorial Educational Society, Hyderabad)
Approved by AICTE & Affiliated to JNTUH
B.Tech Courses offered: CIVIL, CSE, IT, ECE, EEE & MECH, NBA Accredited Courses: CIVIL, CSE, ECE, EEE & MECH, Accorded NAAC A - Grade

DEPARTMENT OF CSE(DATA SCIENCE)

CERTIFICATE

This is to certify that " the Major Project report entitled "***COGNIFY AI Driven Learning, Visualization & Gamification***" is a bonafide work done by ***A Rushmitha. (22AG1A6770),A Shivatmika(22AG1A6765),.R Sindhu Meghana(22AG1A67B5),D Varun Kumar(22AG1A6787)*** in partial fulfillment for the award of degree of BACHELOR OF TECHNOLOGY in CSE (Data Science) from JNTUH University, Hyderabad during the academic year 2025- 2026 is a record of Bonafide work carried out by them under our guidance and supervision.

The results embodied in this report have not been submitted by the student to any other University or Institution for the award of any degree or diploma.

Mrs. P Niharika	**Dr. R Suresh**	**External**
Assistant Professor	Associate Professor	
Supervisor	HOD, CSE-DS	

ACKNOWLEDGEMENT

We would like to express my gratitude to all the people behind the screen who have helped me transform an idea into a real time application. We would like to express my heart-felt gratitude to our parents without whom we would not have been privileged to achieve and fulfill our dreams.

A special thanks to our General Secretary, **Prof. Y. V. Gopala Krishna Murthy,** for having founded such an esteemed institution. Sincere thanks to our Joint Secretary

Mrs. M. Padmavathi, for support in doing project work. We are also grateful to our beloved principal, **Dr. N. Sudhakar Reddy** for permitting us to carry out this project.

We profoundly thank **Dr. R. Suresh**, Associate Professor and Head of the Department of Computer Science and Engineering (Data Science), who has been an excellent guide and also a great source of inspiration to our work.

We extremely thank **Dr. P. Ashok Kumar and Mr. Shaik Nagur Vali,** Assistant Professors, Project coordinators, who helped us in all the way in fulfilling of all aspects in completion of our Major-Project.

We am very thankful to my internal guide **Mrs. P Niharika** who has been an excellent and also given continuous support for the Completion of our project work. The satisfaction and euphoria that accompany the successful completion of the task would be great, but incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success. In this context, we would like to thank all the other staff members, both teaching and non-teaching, who have extended their timely help and eased our task..

A. Rushmitha (22AG1A6770)
A. Shivatmika (22AG1A6765)
R. Sindhu Meghana (22AG1A67B5)
D. Varun Kumar (22AG1A6787)

DECLARATION

We hereby declare that the result embodied in this project report entitled **"COGNIFY AI DRIVEN LEARNING, VISUALIZATION & GAMIFICATION"** was carried out by us during the year 2025-2026 for the partial fulfilment of the award of **Bachelor of Technology in Computer Science and Engineering (Data Science)**, from **ACE ENGINEERING COLLEGE.** We have not submitted this project report to any other Universities/Institute for the award of any degree.

A. Rushmitha (22AG1A6770)

A. Shivatmika (22AG1A6765)

R. Sindhu Meghana (22AG1A67B5)

D. Varun Kumar (22AG1A6787)

COGNIFY AI Driven Learning, Visualization & Gamification

ABSTRACT

In today's digital learning era, traditional teaching methods often fail to keep students engaged or support diverse learning needs. To address this gap, **Cognify** is designed as an AI- powered learning platform that enhances understanding, retention, and interactivity. It uses natural language processing and machine learning to interpret learner inputs, creating a personalized and adaptive learning experience that goes beyond static videos and text- based materials.

Cognify allows users to interact through **text or voice**, which is processed using speech- to- text and advanced NLP algorithms to extract key concepts. These concepts are automatically converted into a wide range of visual formats, including **step-by-step images, flowcharts, process diagrams, concept maps, mind maps, 3D models, and analytical charts**. Along with this, the system also generates **summaries, examples, and quizzes** to reinforce learning, making complex topics easier to understand through clear visual and interactive aids.

To further enhance engagement, Cognify incorporates **gamification elements** such as points, badges, and progress tracking through an analytics dashboard. These features motivate learners, encourage consistency, and provide measurable outcomes. By combining AI-driven visualization, interactive content generation, and gamification, Cognify transforms the learning experience into one that is **immersive, adaptive, and highly effective** for students at any level.

CONTENT

S. No.	CHAPTER NAME	Page No
1.	**INTRODUCTION**	**1-4**
	1.1 ProblemStatement and Objective	
	1.2 Significance and Motivation	
	1.3 Scope of the project	
2.	**LITERATURE SURVEY**	**5-9**
	2.1 Existing Problems	
	2.2 Drawbacks ofExisting System	
	2.3 Proposed System	
	2.4 Advantages ofproposed System	
3.	**REQUIREMENTS ANALYSIS**	**10-12**
	3.1 Hardware Requirements	
	3.2 Software Requirements	
	3.3 Functional Requirements	
	3.4 Non-Functional Requirements	
4.	**SYSTEM ANALYSIS**	**13-17**
	4.1 Methodology	
	4.2 Modules	
5.	**SYSTEM DESIGN**	**18-26**
	5.1 System Architecture	
	5.2 ER Diagrams	
	5.3 Data Flow Diagrams	

6. IMPLEMENTATION 27-45

6.1 Cleaning and EDA

6.2 Feature Engineering

6.3 Model Training

6.4 Model Evaluation

6.5 Results and Discussion

6.6 Source Code

7. OUTPUT SCREENS 45-51

8. CONCLUSION AND FUTURE SCOPE 52-53

REFERENCES 54

LIST OF FIGURES

S.No.	FIGURE NAME	Page No
1.	Methodology	13
2.	System Architechture	18
3.	ER-Diagram	19
4.	Dataflow Diagram	20
5.	Use Case Diagram	22
6.	Class Diagram	23
7.	Sequence Diagram	24
8.	Activity Diagram	25
9.	Deployment Diagram	26
10.	Output Screens	45-51

1. INTRODUCTION

1.1 Problem Statement & Objectives

In today's digital learning environment, many students struggle to understand and remember complex concepts when relying on traditional methods such as lengthy paragraphs, textbooks, and long lectures. These methods often lack meaningful visualization, making it difficult for learners to follow processes, connect ideas, and retain important information. Since not all students learn in the same way or at the same pace, this mismatch between teaching format and learning needs results in confusion, reduced interest, and difficulty in applying concepts effectively.

To address these challenges, there is a growing need for an intelligent system that can convert complex educational content into easy-to-understand visual formats while adapting to the learner's individual style. Cognify aims to bridge this gap by transforming text or voice- based queries into interactive and clear visuals such as flowcharts, mind maps, step-by-step diagrams, concept maps, and even 3D models. The platform also offers personalized explanations, summaries, and examples based on the learner's understanding level, ensuring that each student receives guidance suited to their specific learning preferences.

Furthermore, Cognify focuses on improving engagement and retention by including interactive visuals and gamification elements such as quizzes, points, and badges. These features make learning more enjoyable and motivate students to stay consistent in their study habits. By presenting information visually and interactively, the platform enhances the learner's ability to recall and apply concepts, making the overall learning experience more effective, engaging, and personalized.

1.1.1 Significance and Motivation

The significance of this project lies in addressing the growing need for effective, interactive, and visually rich learning support in modern education. Today's academic environment expects students to quickly understand complex concepts, apply knowledge in real situations, and retain information for long periods. However, traditional learning resources—text-heavy notes, long lectures, and static diagrams— often fail to meet these expectations. They provide limited visualization and offer little personalization, making it difficult for many students to follow ideas clearly or stay engaged. Cognify tackles these limitations by using AI to convert text or voice inputs into meaningful visual formats such as flowcharts, mind maps, step-by-step diagrams, concept maps, and even 3D models, allowing learners to grasp topics more quickly and effectively.

The platform's significance is further highlighted by its ability to offer personalized learning support tailored to each student's pace, preferences, and level of understanding. Through AI-driven explanations, real-time assistance, and adaptive examples, Cognify ensures that learners receive guidance suited to their needs. Its interactive visuals also enhance retention, making concepts easier to recall compared to plain text. Additionally, Cognify supports educators through analytical dashboards that help track student progress, identify strengths and weaknesses, and refine teaching strategies. This makes the platform valuable not just for students but also for teachers and institutions seeking smarter learning solutions.

The motivation behind Cognify stems from the increasing demand for learning tools that are more intuitive, engaging, and aligned with how today's students prefer to learn. Many learners struggle with lengthy textual explanations, lack of visual guidance, low motivation during traditional lectures, and the absence of personalized help. With advancements in AI and NLP, there is a strong opportunity to overcome these challenges by transforming complex information into simple, understandable visuals instantly. Cognify is driven by the goal of making learning easier, more enjoyable, and more accessible for all learners. It aims to close the gap between difficult academic content and student comprehension, support self-learning through instant visual responses, encourage continuous learning through gamified elements, and promote equal learning opportunities regardless of a student's background or learning speed.

1.2 Scope of the Project

The scope of the Cognify project covers the development of an AI-powered visual learning platform that transforms complex educational content into simple, interactive, and personalized visuals. The project focuses on enhancing the learning experience for students and supporting teachers with insightful tools.

1.2.1 Content Processing and Understanding

1.2.1.1 The system will accept text or voice-based queries from learners.

1.2.1.2 It will use AI and NLP to analyze the content and understand the key concepts.

1.2.1.3 The platform will support multiple subjects and academic topics.

1.2.2 Visual Content Generation

1.2.2.1 Convert complex concepts into flowcharts, mind maps, step-by-step diagrams, concept maps, and 3D models.

1.2.2.2 Provide clean, structured, and easy-to-understand visuals for better comprehension.

1.2.2.3 Ensure visuals are auto-generated based on user input.

1.2.3 Personalized Learning

1.2.3.1 Offer summaries, explanations, and real-life examples based onthe learner's understanding level.

1.2.3.2 Adapt responses to different learning styles—visual, step-by-step, or conceptual.

1.2.3.3 Provide instant clarification for student queries.

1.2.4 Gamification Features

1.2.4.1 Include gamified elements such as quizzes, points, levels, and achievement badges.

1.2.4.2 Motivate students to stayengaged and track their learning streak.

1.2.5 Dashboard and Progress Tracking

1.2.5.1 Provide separate dashboards for students and teachers.

1.2.5.2 Track performance metrics like quiz scores, topic mastery, and learning progress.

1.2.5.3 Help teachers identify strengths, weaknesses, and required improvements.

1.2.6 User Interaction and Accessibility

1.2.6.1 Provide a user-friendly interface that works on web and mobile platforms.

1.2.6.2 Allow students to interact through typing or speaking.

1.2.6.3 Support multilingual capabilities in future extensions.

1.2.7 Data Storage and Management

1.2.7.1 Store generated visuals, student progress, quizzes, and user profiles securely.

1.2.7.2 Maintain structured databases for content, user activity, and analytics.

2. LITERATURE SURVEY

The literature survey shows that existing systems focus on personalized text learning, manual visual aids, gamification, or adaptive tutoring, but none combine these features into a single AI-driven platform. Current tools lack automatic visual generation, real-time concept mapping, and seamless integration of quizzes and analytics. This gap highlights the need for an advanced system like Cognify.

1. Sinha et al. (2023)

   Analyzed AI-based personalized learning tools like Khanmigo and Duolingo GPT- 4, but these systems primarily adapt text-based content and do not generate visual learning materials such as diagrams or concept maps.

2. Lee et al. (2019)

   Highlighted the effectiveness of visual aids such as concept maps and flowcharts, though their study focuses on manual visualization methods and does not include AI-driven automatic visu.al generation.

3. Patel et al. (2020)

   Examined gamification elements that enhance learner motivation, yet the approach lacks integration with personalized AI support or real-time visual learning features.

4. Woolf et al. (2021)

   Reviewed intelligent tutoring systems offering adaptive feedback, but existing ITS solutions rely on predefined content and do not convert student queries into dynamic, AI-generated visuals.

2.1 Existing Problems

- Traditional E-Learning Platforms (Coursera, EdX, Udemy)
- Deliver video lectures, quizzes, and static materials, offering limited visualization or interactive learning support.
- Gamified Learning Apps (Duolingo, Kahoot)
- Provide reward-based learning with points, badges, and streaks but focus mainly on languages and do not support conceptual visualization.
- Mind Mapping & Diagramming Tools (MindMeister, Lucidchart)
- Enable users to create diagrams manually but do not generate visual explanations automatically from text or voice queries.
- Single Image Generation Tools (Canva AI, Bing Image Creator)
- Generate simple static images but cannot create structured educational visuals such as flowcharts, step-by-step diagrams, concept maps.

2.2 Drawbacks of Existing System

- Lack of Personalization: Current systems rarely adapt to each student's individual learning style or pace.
- Limited Visualization: Most platforms depend on static text or video lessons, which make it hard to visualize complex topics.
- Manual Effort Required: Tools that allow diagram creation need manual work instead of using AI to auto-generate visuals.
- Low Interaction: Students mainly consume information passively rather than interacting or asking questions dynamically.
- No Real-Time Support: Existing systems do not instantly convert text or voice queries into understandable visuals or explanations
- Limited Engagement: Without enough gamified elements, learners may lose interest or motivation over time.
- No Real-Time Support: Existing systems do not instantly convert text or voice queries into understandable visuals or explanations
- Limited Engagement: Without enough gamified elements, learners may lose interest or motivation over time.

2.3 Proposed System

- AI & NLP Processing: Analyzes bothtext and voice inputs to extract concepts, keywords, and important details using advanced NLP models.
- Dynamic Visualizations: Creates auto-generated flowcharts, mind maps, concept maps, process diagrams, and 3D models to simplify complex topics.
- Interactive Reinforcement: Produces quizzes, summaries, hints, and real-time examples to reinforce understanding.
- Gamification: Includes badges, points, levels, streaks, and leaderboards to keep users motivated and engaged.
- Progress Analytics: Offers personalized dashboards to track quizzes, time spent, learning streaks, and weak areas.
- Speech-to-Text Conversion: Converts voice input into text using Whisper to provide hands- free learning.
- Content Personalization: Tailors visuals, quiz difficulty, and explanations based on user learning patterns.
- Offline Functionality: Works locally using Python and JSON-based file storage, enabling usage without internet access.
- Error Handling & Feedback: Provides suggestions, corrections, and feedback when unclear or incomplete input is given.
- Data Storage & Retrieval: Stores summaries, quizzes, images, and session logs securely for later review.

2.4 Advantages of proposed System

- Enhances understanding byconverting complex text into clear visualdiagrams and summaries.
- Supports diverse learners including beginners, slow learners, and visual learners.
- Reduces cognitive load by breaking long topics into simple diagrams and key points.
- Improves engagement through interactive quizzes and gamified rewards.
- Boosts accuracyand efficiencywith AI-generated content that is consistent and error-free.
- Provides personalized growth tracking to help learners understand their strengths and weaknesses.
- Saves facultyeffort by automatically generating visuals and assessments for teaching.
- Works offline, making it useful for students without continuous internet access.
- Encourages continuous learning with rewards, streaks, and progress monitoring.
- Highly scalable, allowing future expansion with more visualization types or advanced ML models.

3. REQUIREMENTS ANALYSIS

3.1 Hardware Requirements

The software requirements for Cognify include Python as the primary programming language along with essential AI and NLP libraries for text processing, summarization, and speech-to- text using Whisper. The user interface is developed using PyQt5, providing an interactive desktop application experience. Visualization is generated using Graphviz and other Python- based rendering tools, while JSON-based file storage is used as the local database for storing user sessions, summaries, images, and quiz data. Additional tools such as VS Code or PyCharm and the pip package manager support the development and deployment of the system.

- Operating System: Windows 10 / Windows 11
- Programming Languages: Python 3.x, HTML, CSS, JavaScript
- Frameworks: Flask or Streamlit
- NLP Libraries: SpaCy, TextRank, TF-IDF
- AI Models: Whisper (Speech-to-Text), Transformers
- Visualization Tools: Graphviz, Matplotlib, NetworkX, PIL
- Database: JSON-based file storage
- Development Tools: VS Codeor PyCharm
- Version Control: Git & GitHub
- Package Manager: pip
- Other Tools: Python Virtual Environment, Web Browser (Chrome/Firefox)

3.2 Software Requirements

The hardware requirements for Cognify include a basic computing system capable of running Python applications efficiently. A device with at least an Intel i3 processor or equivalent, a minimum of 4GB RAM (8GB recommended for faster AI and visualization processing), and at least 10GB of free disk space is needed for storing libraries, models, and output files. A microphone is required for voice-based input, while a standard display, keyboard, and mouse support interaction with the PyQt5 interface. These specifications ensure smooth performance for NLP processing, visualization generation, and database operations. JSON-based storage is used to store user sessions, summaries, images, and quiz data locally.

- **Processor:** Intel i3 or equivalent (Intel i5 recommended)
- **RAM:** Minimum 4GB (8GB recommended for faster processing)
- **Storage:** At least 10GB free disk space
- **Input Devices:** Keyboard, Mouse, Microphone (for voice input)
- **Display:** Standard monitor or laptop screen

3.3 Functional Requirements

- **Input Layer:** Accepts text/voice input, performs speech-to-text with Whisper, and stores input in JSON files.
- **Summary Generation:** Cleans input and generates summaries, keypoints, and keywords using TextRank and TF-IDF.
- **Visualization Engine:** Converts concepts into flowcharts, mind maps, concept maps, process diagrams, charts, and 3D models.
- **Quiz Generation:** Creates MCQs, tracks scores, and stores quiz results in the quiz_attempts table.
- **Gamification & RL:** Awards points, badges, and levels while dynamically adapting difficulty using reinforcement learning.
- **Database & Persistence**: Uses JSON files to store users, sessions, summaries, images, and achievements.

3.4 Non-Functional Requirements

- **Usability:** Interface should be simple, interactive, and user-friendly.
- **Performance:** Visualization and NLP processing should respond within 3–5 seconds for typical queries.
- **Scalability:** Should support multiple concurrent users.
- **Security:** Data and user activitis must be protected using authentication and encryption.
- **Portability:** Should run on different platforms (Windows, macOS, Linux).
- **Maintainability:** System modules should be modular and easilyupgradable.
- **Reliability:** The system should run consistently without failures during normal use.
- **Availability:** The system must be accessible to users whenever required.

4. SYSTEM ANALYSIS

The system takes user input through text or voice, processes it with NLP, and generates summaries and visual diagrams. It then creates automatic quizzes, applies gamification, and predicts user performance. All activities are tracked through an analytics dashboard and stored securely in structured JSON files

Methodology:

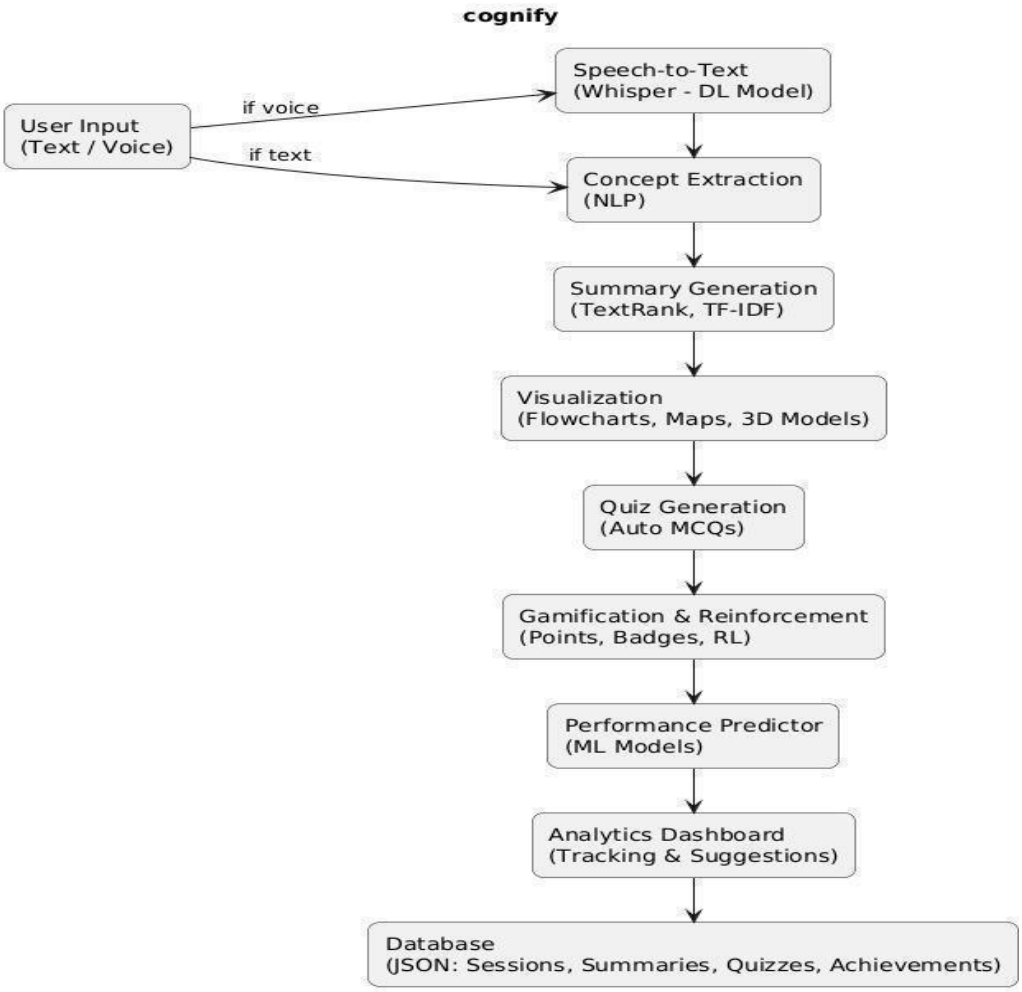

Fig 1. Methodology

1. User Input

- Learners provide queries through either text or voice input.

  If the input is voice, it is converted into text using a Speech-to-Text module (Whisper - Deep Learning Model).

- The processed text serves as the foundation for further analysis.

2. Concept Extraction (NLP)

- Using Natural Language Processing (NLP), the system extracts keywords, entities, and main concepts from the input.
- This step helps identifythe core ideas and relationships between terms.

3. Summary Generation

- The extracted concepts are summarized using AI-based algorithms such as TextRank and TF-IDF.
- The result is a concise overview that highlights keypoints and essential information.

4. Visualization Generation

- The summarized and structured content is converted into visual representations like flowcharts, mind maps, data charts, and 3D models.
- Tools such as Graphviz and AI layout algorithms are used to generate these dynamic visuals.

5. Quiz Generation

- Fromthe extracted concepts, the system automatically generates multiple-choice questions (MCQs) and short quizzes.
- This helps in reinforcing learning and evaluating understanding.

6. Gamification & Reinforcement

- Learners earn points, badges, and rewards for completing activities or achieving milestones.
- A Reinforcement Learning (RL) model adapts the difficulty and recommends personalized challenges based on performance.

7. Performance Predictor

- Using Machine Learning models (like Logistic Regression or Random Forest), the system predicts the learner's future performance.
- It analyzes quiz history, time spent, and accuracytrends to estimate progress.

8. DataStorage (JSON)

- Alluser interactions, summaries, quizzes, and achievements are stored securely in local JSON files
- This ensures persistent tracking and offline accessibility.

4.1 Modules

4.1.1 User Input and Interaction Module

This module handles all user input through both text and voice. Text is received directly, while voice input is processed through a Speech-to-Text engine using the Whisper deep- learning model. The converted text becomes the foundation for NLP processing. This module ensures seamless interaction, enabling learners to communicate naturally with the system through either typing or speaking.

4.1.2 NLP Concept Extraction Module

This module uses Natural Language Processing techniques to extract keywords, key entities, and core concepts from the user's query. By identifying relationships between terms, it prepares structured information for summarization and visualization. This module forms the analytical core that interprets the user's intent.

4.1.3 Summary Generation Module

This module processes the extracted concepts using algorithms such as TextRank and TF- IDF to generate concise summaries. It produces key points, keywords, and simplified explanations that help users understand complex topics more easily. The generated summary becomes input for visual and quiz modules.

4.1.4 Visualization Generation Module

This module converts summarized content into useful visual formats like flowcharts, mind maps, concept maps, data charts, and 3D models. Using Graphviz and Python-based layout algorithms, it automatically creates dynamic and easily understandable diagrams. These visuals help learners grasp concepts quickly.

4.1.5 Quiz Generation Module

This module automatically generates multiple-choice questions (MCQs) based on the extracted and summarized content. It evaluates user responses, assigns scores, and records quiz attempts. The quiz engine reinforces learning and assesses user understanding in real time.

4.1.6 Gamification and Reinforcement Learning Module

This module motivates learners by awarding points, badges, and achievement levels. A reinforcement learning model adapts the difficulty of tasks based on user performance, ensuring personalized progression. The module enhances engagement, retention, and long- term learning effectiveness.

4.1.7 Performance Prediction and Analytics Module

This module uses machine learning models such as Logistic Regression or Random Forest to analyze quiz history, time spent on tasks, accuracy trends, and overall interaction patterns. It predicts the learner's future performance and provides insights through dashboards, allowing users to track their growth and areas of improvement.

4.1.8 Database and Storage Management Module

This module manages all data storage using JSON-based files, including user sessions, summaries, quizzes, images, points, badges, and achievements. It ensures secure data handling, fast retrieval, and offline accessibility. Structured JSON objects for efficient data organization support efficient System Performance.

5. SYSTEM DESIGN

5.1 System Architecture

The System Architecture of Cognify follows a modular layered design consisting of a User Interaction Layer for handling text and voice input, a Processing and Intelligence Layer that performs NLP, summarization, visualization, quiz generation, and gamification, and a Data Management Layer that stores all user data and learning activities in JSON files. These layers work together smoothly to convert user queries into meaningful visual explanations and personalized learning outputs.

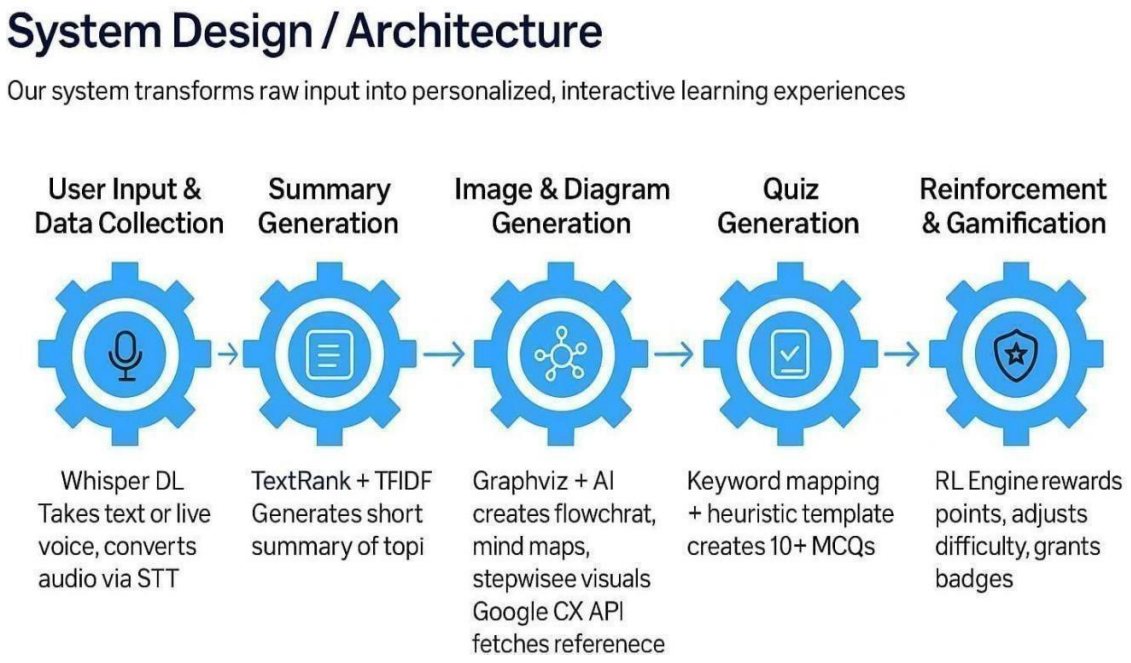

Fig 2. System Architecture

5.2 ER-Diagram

The ER Diagram of the Cognify platform centers around the User entity, which interacts with the system by submitting text or voice-based queries stored in the User Query entity. These queries are processed by AI components and linked to the Visualization Output entity, which contains various visual learning materials such as flowcharts, mind maps, step-by-step diagrams, concept maps, and 3D models. This relationship ensures that each visual output is directly connected to the query that generated it, maintaining a clear and personalized learning flow.

The diagram also includes the Learning Progress and Gamification Module entities, which record user activities such as completed topics, scores, badges, and interaction history. These modules work together to enhance engagement and track user performance over time. By connecting User, User Query, Visualization Output, Learning Progress, and Gamification elements, the ER diagram illustrates how Cognify delivers an integrated system that converts inputs into meaningful visuals, monitors progress, and supports personalized, interactive learning.

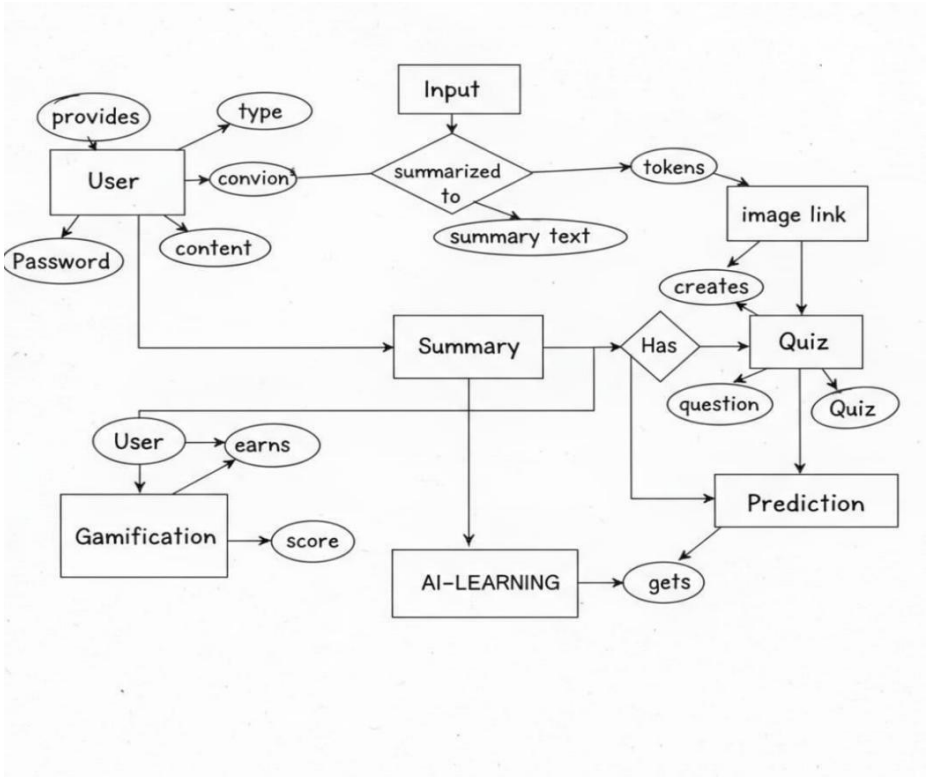

Fig 3. ER Diagram

5.3 Data Flow Diagram

The Data Flow Diagram shows how Cognify processes user input from text or voice. Voice input is converted to text using the Whisper Speech-to-Text module. The text then passes through the Summary Generation module, which creates concise explanations using TextRank and Wikipedia data. Next, the Image Generation module produces visual diagrams, followed by the Quiz Generation module that creates MCQs. Finally, all outputs—including summaries, images, and quiz data—are saved in JSON storage files. All modules run locally using Python.

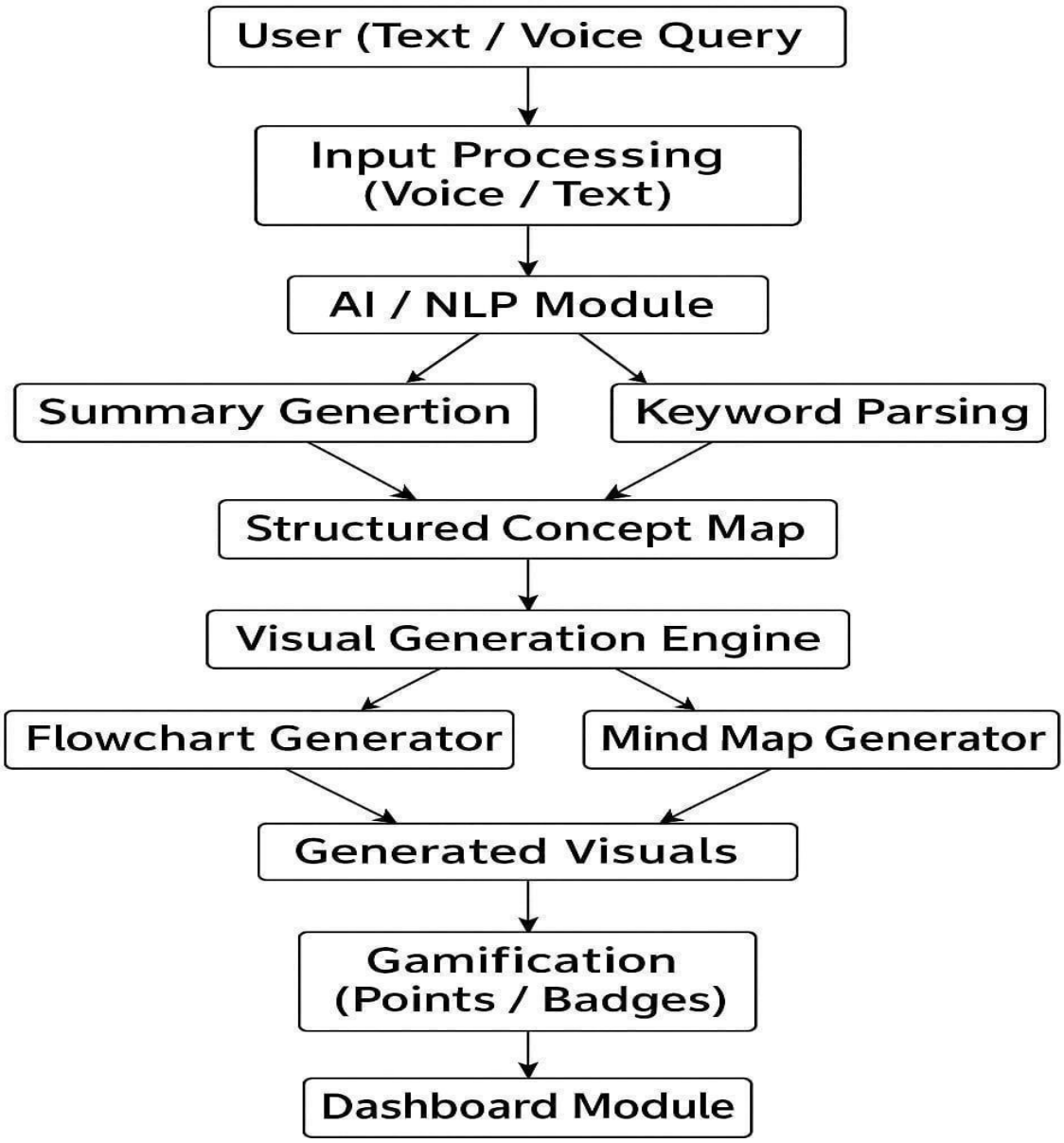

Fig 4. Data flow Diagram

5.4 UML Design

Unified Modeling Language (UML) is a standardized visual modeling method used to design, analyze, and document software systems. It provides a collection of diagrams that represent a system's structure, behavior, and interactions in a clear graphical format. UML helps simplify complex software by breaking it into understandable components, making it easier for developers and stakeholders to visualize system functionality.

In the Cognify project, UML is used to illustrate how the platform processes text and voice inputs, performs NLP-based concept extraction, generates summaries and visual diagrams, creates quizzes, handles gamification, and manages data storage. These diagrams give a clear understanding of how the AI-driven modules interact within the system's layered architecture, ensuring proper design and smooth workflow.

UML also maintains consistencythroughout development by offering a common visual language for the entire project team. It reduces confusion by clearly defining responsibilities, data flow, and module interactions. This helps ensure that the development process remains structured, efficient, and aligned with project goals.

For Cognify, UML diagrams highlight the modular nature of the system, showcasing how the Input Module, NLP Engine, Summary Generator, Visualization Engine, Quiz Generation Module Gamification system and JSON-based storage system connect and communicate These diagrams support implementation, simplify debugging, and enable future enhancements, ultimately making Cognify more scalable, maintainable, and easy to understand.

5.4.1 USE CASE Diagram

The Use Case Diagram represents the functional interactions between the users and the Cognify system. It identifies how different actors communicate with the system and what major services are offered to them. In Cognify, the primary actor is the Student, who interacts with the system to submit queries, view generated summaries and visualizations, take quizzes, and track progress. The Admin manages the system, updates learning models, and maintains user data. This diagram helps in understanding the functional scope of the system from an end- user perspective.

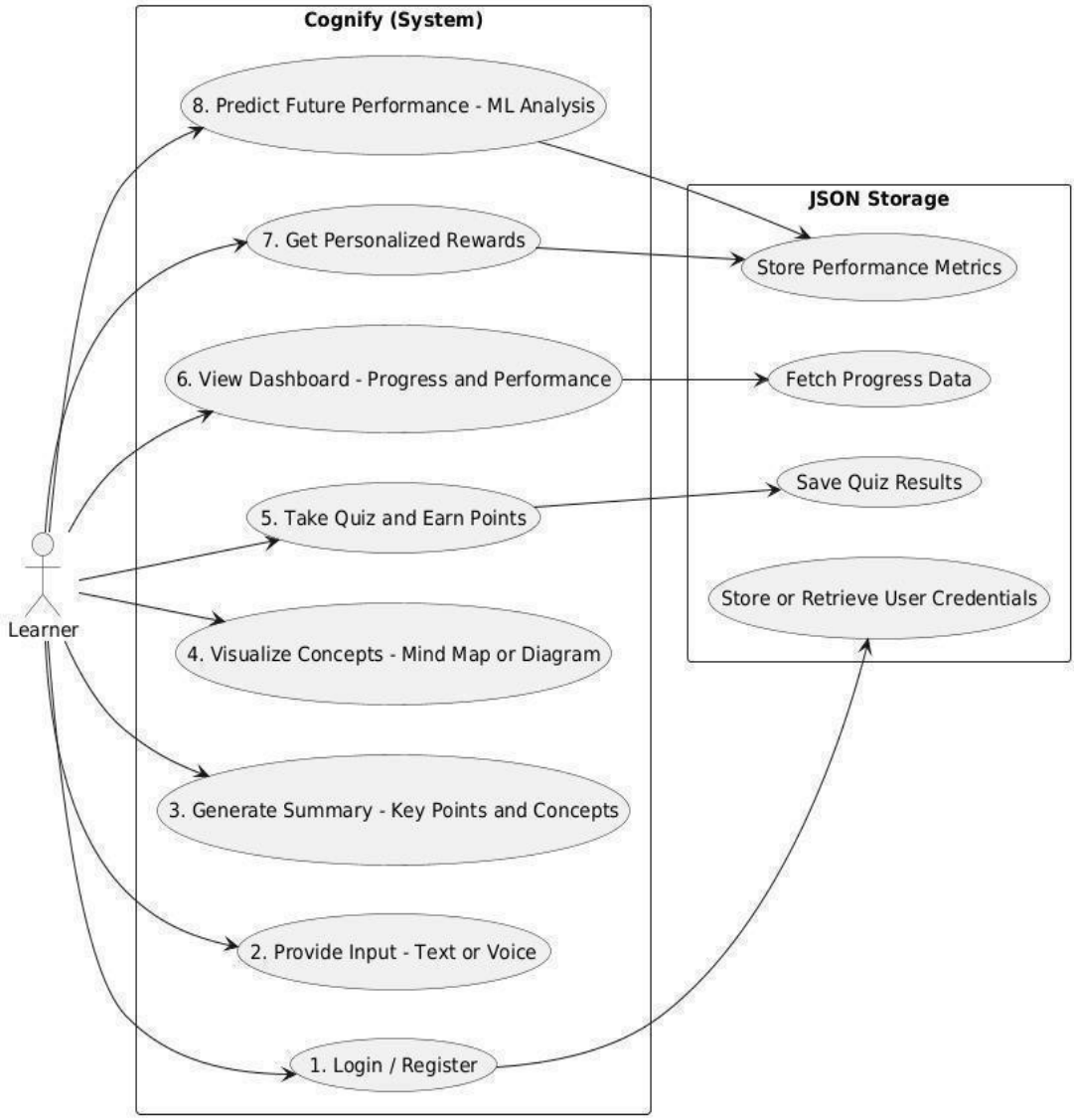

Fig 5. UseCase Diagram

5.4.2 Class Diagram

The Class Diagram illustrates the static structure of Cognify by depicting its main classes, their attributes, methods, and relationships. It represents how the system is organized internally and how different components interact. Major classes such as User, InputHandler, NLPProcessor, SummaryGenerator, VisualizationEngine, QuizGenerator, and DatabaseManager work together to process input, generate diagrams, evaluate quizzes, and maintain data. This diagram shows the blueprint of the system's architecture, inheritance, associations, and dependencies.

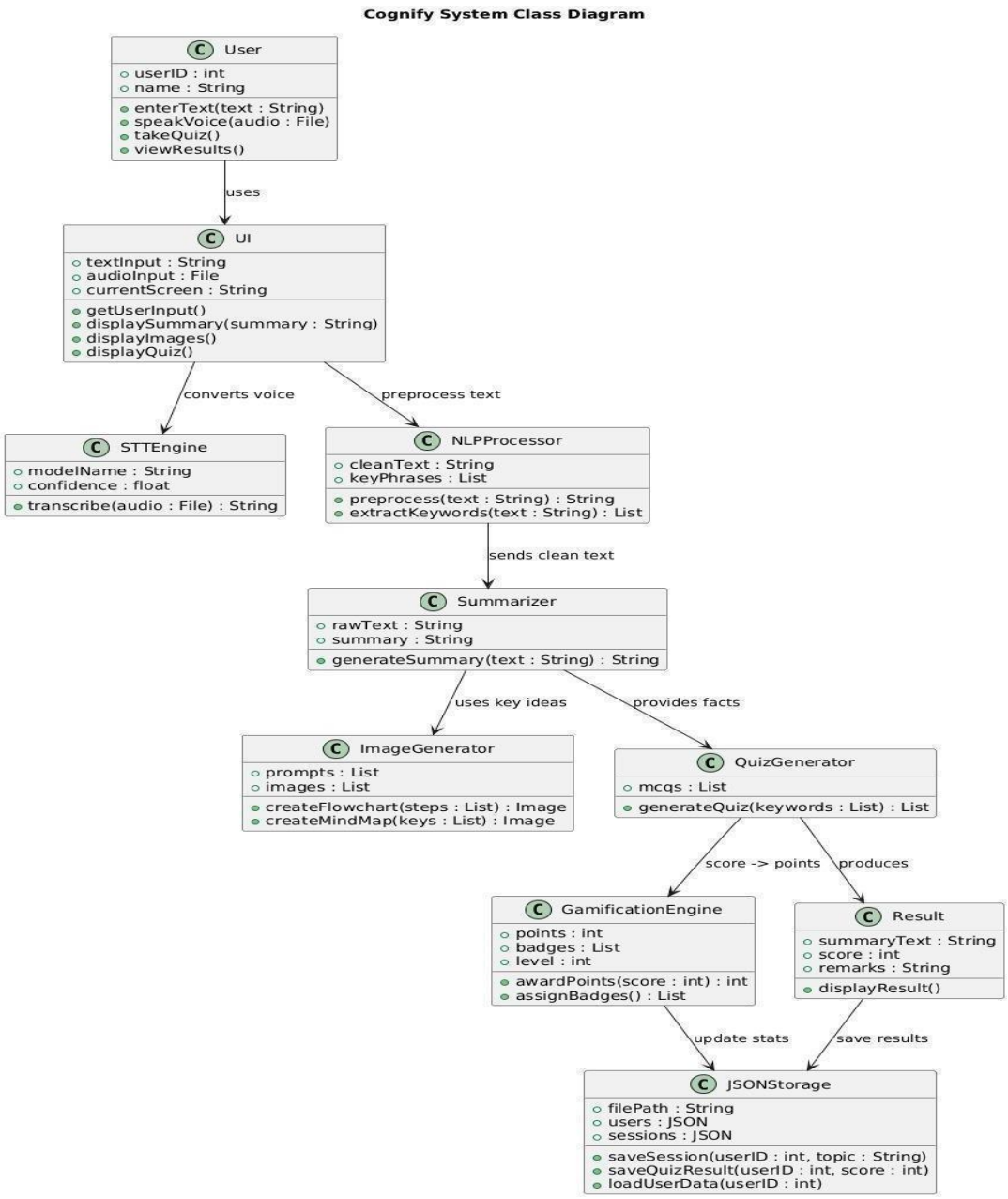

Fig 6. Class Diagram

5.4.3 Sequence Diagram

The Sequence Diagram explains the step-by-step flow of operations within the Cognify system. It highlights how objects communicate and in what order. The flow begins when a student submits a text or voice input, which is processed by the NLP engine to generate summaries and visuals. The student can also request quizzes, and the system evaluates answers, updates gamification rewards, and stores progress. This diagram helps visualize the time-based interaction between system components.

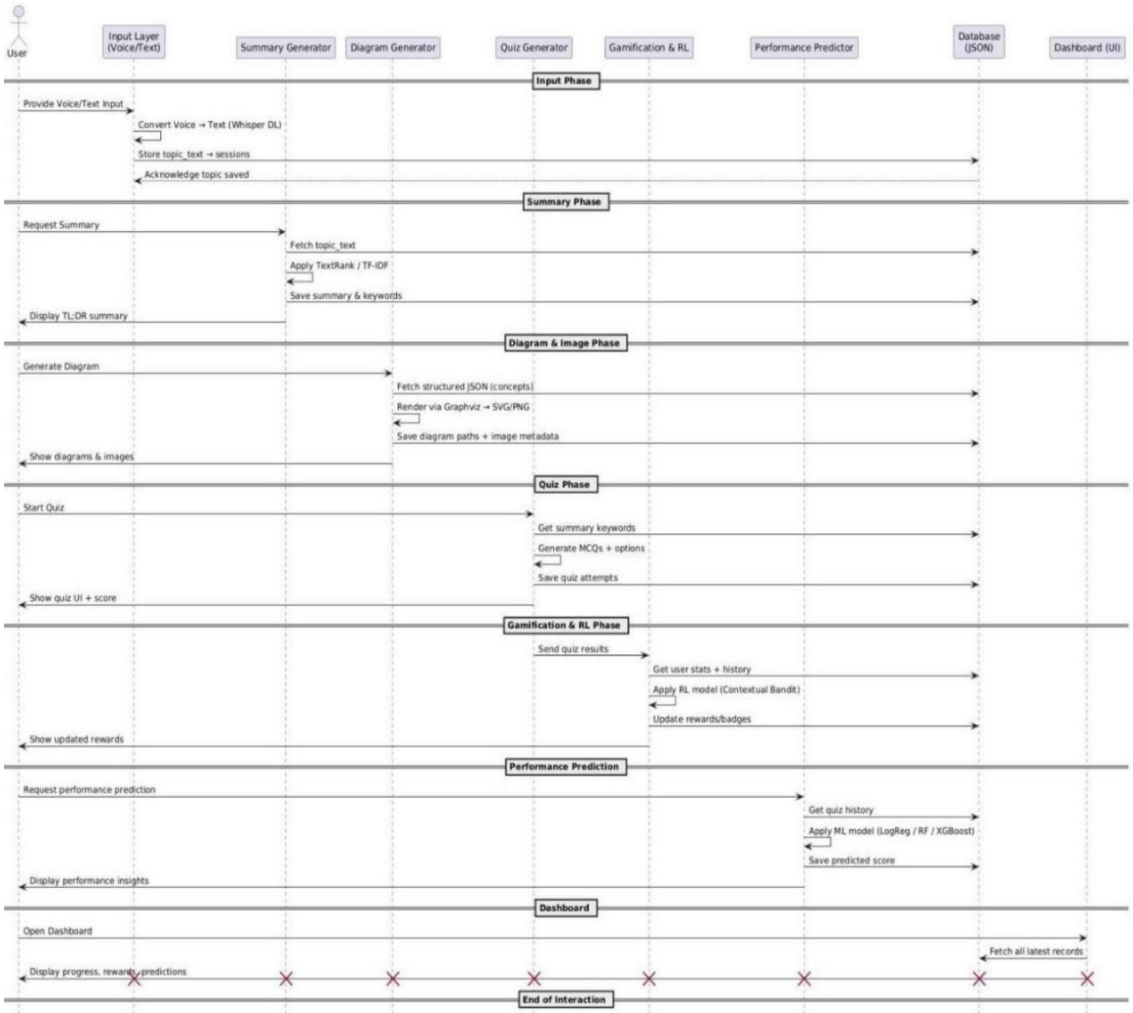

Fig 7. Sequence Diagram

5.4.4 Activity Diagram

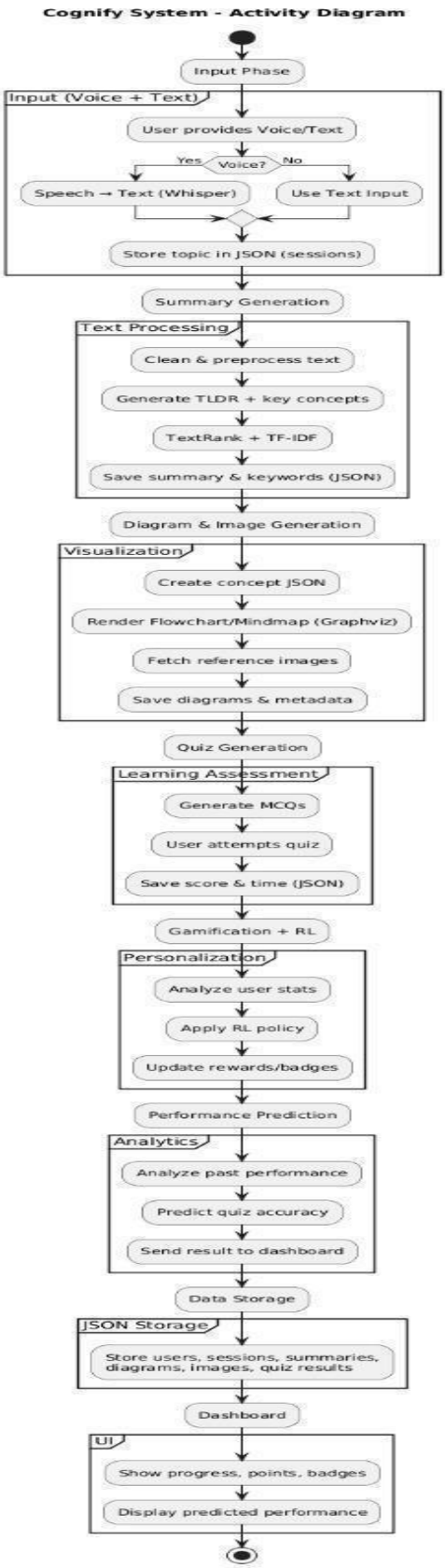

Fig 8. Activity Diagram.

5.4.5 Deployment Diagram

The Deployment Diagram shows the physical arrangement of hardware and software components used by the Cognify system. It includes three main nodes: the User Device, Application Server, and Database Server. The application server hosts all core modules such as NLP processing, visualization generation, quiz engine, and gamification logic, while the data storage module manages JSON files for storing users, sessions, quiz data, and summaries. This diagram demonstrates how the system is deployed across different environments and how communication occurs between nodes.

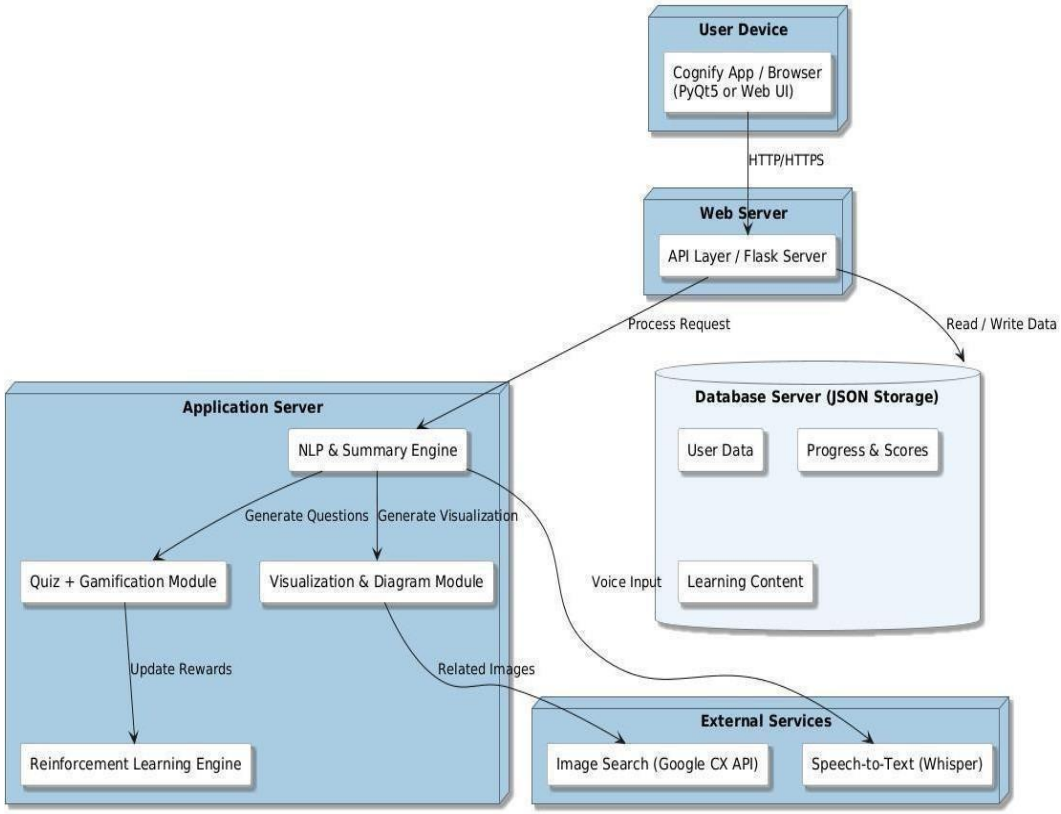

Fig 9. Deployment Diagram

6. IMPLEMENTATION

6.1 Cleaning and Exploratory Data Analysis (EDA)

The data acquisition phase of Cognify AI involved collecting structured and semi- structured learning interaction data generated from user sessions, including topic inputs, quiz scores, time spent per topic, number of attempts, difficulty levels, and engagement metrics. Since user-generated data often contains inconsistencies, noise, and irregular formatting, a comprehensive preprocessing pipeline was implemented to ensure data integrity and analytical reliability. Raw inputs frequently contained irregular whitespace, inconsistent capitalization, duplicate entries, missing values, and transcription noise introduced during speech-to-text conversion. Using python libraries such as pandas, re, nltk, and json, the system performed systematic cleaning operations including duplicate removal to prevent biased modeling, handling missing values through statistical imputation techniques such as mean and median substitution, and normalization of textual data by converting to lowercase and removing punctuation. Stopword removal and lemmatization were applied to standardize word forms and reduce semantic redundancy. Invalid numerical entries, such as quiz scores exceeding logical thresholds, were filtered to maintain data validity, and outliers were detected using interquartile range (IQR) analysis to ensure distribution consistency. For textual topic inputs, tokenization and normalization ensured semantic uniformity before further processing in the Natural Language Processing pipeline. This structured cleaning framework improved dataset quality, reduced noise, and ensured that subsequent analytical and predictive processes operated on high-quality, consistent data.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\s]", " ", text)

def preprocess_text(text):
    tokens = tokenize_and_filter(cleaned)
    lemmas = lemmatize_tokens(tokens)
```

Before:

```
Machine learning is a method of data analysis that automates analytical model building.
It is a branch of artificial intelligence based on the idea that systems can learn from data,
identify patterns and make decisions with minimal human intervention.
```

After:

```
machine learning method data analysis automate analytical model building
branch artificial intelligence base idea system learn data identify pattern make decision minimal
```

Exploratory Data Analysis:

Exploratory Data Analysis (EDA) was conducted to examine learning behavior patterns and uncover relationships among variables prior to model training. Using visualization libraries such as Matplotlib and Seaborn, the system analyzed the distribution of quiz scores, correlation between time spent and performance, variation in scores across difficulty levels, and engagement trends across repeated attempts. Histogram analysis revealed the overall score distribution and highlighted performance clustering patterns among users. Correlation heatmaps were used to measure relationships between engagement score, number of attempts, time spent, and quiz performance. The analysis indicated a moderate positive correlation between engagement score and quiz performance, validating the relevance of engineered engagement features. Additionally, comparative analysis across difficulty levels revealed increased performance variance at higher difficulty settings, emphasizing the importance of adaptive learning mechanisms. Trend analysis of repeated attempts demonstrated progressive improvement patterns, supporting the inclusion of temporal features in predictive modelling. EDA provided critical insights for feature selection, reinforcement threshold calibration, anomaly detection, and behavioral understanding, thereby forming the analytical foundation for the machine learning and adaptive learning components of the system.

6.2 Feature Engineering:

Feature engineering in Cognify focused on transforming raw interaction data into meaningful predictive variables capable of improving model performance and interpretability. Instead of relying solely on traditional vectorization methods, the system constructed structured features derived from user behavior, performance trends, and learning patterns. Engagement- related features were generated by combining time spent per topic and number of attempts into an engagement score, which captured user involvement intensity. Performance-based features such as performance ratio, calculated as quiz score divided by attempts, and improvement rate, derived from consecutive score differences, were introduced to measure learning progression. Difficulty levels were encoded numerically to allow machine learning algorithms to interpret categorical information effectively. Furthermore, a temporal learning trajectory feature was synthesized to represent skill evolution over time, enabling the dashboard to visualize longitudinal performance growth rather than static single-session scores. A key engineered feature was the Knowledge Gap Index, computed by comparing predicted performance against expected baseline proficiency thresholds for each topic. This comparison produced a gap vector that identified weak areas and underperforming concepts, allowing the system to recommend targeted revision strategies. These engineered features enhanced model robustness, reduced noise from raw variables, and introduced a behavioral dimension to the predictive analysis, ultimately improving personalization and adaptive learning outcomes.

6.3 Model Training:

The predictive component of Cognify was implemented using a Random Forest Regressor from the Scikit-learn library, chosen for its ability to handle non-linear relationships, reduce overfitting through ensemble learning, and provide feature importance insights. Rather than training deep neural networks that require extensive computational resources, the system leveraged the interpretability and efficiency of ensemble decision trees to predict user quiz performance based on engagement, difficulty, and engineered behavioral features. The dataset was split into training and testing subsets using an 80–20 ratio to ensure unbiased evaluation. The model was trained using multiple decision trees aggregated through bootstrap sampling, and

predictions were evaluated using metrics such as Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) to measure accuracy and stability. Feature importance analysis revealed that engagement score and performance ratio were among the most influential predictors confirming insights derived from EDA.

In addition to supervised learning, a reinforcement-based adaptation mechanism was incorporated to dynamically adjust quiz difficulty. Instead of implementing a full-scale deep reinforcement learning architecture, a reward-based adaptive logic was employed, where higher quiz scores resulted in increased difficulty levels, moderate scores maintained the current level, and lower scores triggered a reduction in difficulty. This adaptive feedback loop simulated intelligent tutoring behavior by ensuring that users were neither under-challenged nor overwhelmed, thereby maintaining optimal cognitive engagement throughout the learning process.

6.4 Model Evaluation:

Model evaluation in Cognify focused on predictive accuracy, adaptability, responsiveness, and contextual relevance of generated educational content. Predictive performance was assessed using regression metrics including MAE and RMSE, which demonstrated stable and low-error predictions across varying difficulty levels. The reinforcement adaptation mechanism was evaluated through simulated multi-session learning scenarios, where adaptive difficulty adjustments resulted in smoother performance curves and reduced learning volatility compared to static difficulty systems. Latency performance was also measured to ensure real-time interactivity; optimization of NLP processing pipelines and model inference reduced average response time to under three seconds per interaction, making the system suitable for practical deployment. Summary generation quality was evaluated based on coherence, relevance, and redundancy reduction, with the hybrid TF-IDF and TextRank approach consistently producing concise and contextually aligned summaries. Quiz generation was validated to ensure domain- specific alignment with user topics, demonstrating contextual awareness rather than generic question formulation. The combined evaluation results indicated that the system achieved reliable predictive performance while maintaining responsiveness and adaptive personalization.

6.5 Results and Discussion:

The final implementation of Cognify resulted in a comprehensive adaptive learning platform capable of transforming static topic input into an interactive and intelligent educational experience. The system successfully generated concise summaries, visual mind maps and flowcharts, topic-aligned quizzes, predictive performance insights, and knowledge gap analyses within a unified dashboard interface. The integration of machine learning-based performance prediction and reinforcement-based difficulty adjustment significantly enhanced personalization. compared to traditional static e-learning platforms.

The Random Forest model demonstrated consistent predictive reliability without requiring complex deep learning infrastructure, while the engineered engagement and temporal features enabled meaningful behavioral insights. The adaptive mechanism improved retention and learning stability by continuously calibrating difficulty levels based on user performance. Unlike conventional systems that rely solely on keyword-based analytics, Cognify integrates semantic understanding, statistical modeling, and adaptive logic to deliver holistic learning intelligence. Although minor variability in prediction may occur due to behavioral fluctuations, the system's ability to analyze engagement patterns and dynamically adjust instructional complexity establishes it as a scalable and intelligent educational framework capable of supporting personalized knowledge acquisition and long-term skill development.

6.6 SOURCE CODE:

```
import warnings
import streamlit
as st import os
import json
import
urllib.parse
import
requests
import re
import time
    from PIL import Image,
    ImageDraw from io import
    BytesIO
    from google import genai
    from google.genai import
    types import graphviz
    from dotenv import
    load_dotenv try:
from streamlit_oauth import OAuth2Component
OAUTH_AVAILABLE = True
        except ImportError:
OAUTH_AVAILABLE = False

from streamlit_mic_recorder import speech_to_text MIC_AVAILABLE =
True
except ImportError:
MIC_AVAILABLE = False

from duckduckgo_search import DDGS
DDGS_AVAILABLE = True
except ImportError:
DDGS_AVAILABLE = False
```

```
  warnings.filterwarnings("ignore",
message=".*missing ScriptRunContext.*")
load_dotenv()
CLIENT_ID = "223087762298-
gabegnstefjei1nf9o7den8au6ttrpk9.apps.googleusercontent.com"
CLIENT_SECRET = "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"
REDIRECT_URI  = os.environ.get("OAUTH_REDIRECT_URI",
"http://localhost:8501").strip() GOOGLE_SCOPES                                    =
["openid","https://www.googleapis.com/auth/userinfo.email","https://www.google
apis.com/auth/
userinfo.profile"]
AUTHORIZE_URL =
"https://accounts.google.com/o/oauth2/auth"  TOKEN_URL
                    =
"https://oauth2.googleapis.com/token"
ADMIN_EMAIL  =
"rushmithaarelli05@gmail.com"  ADMIN_PASS
                                = "Cognify@Admin2025"
USERS_FILE = "cognify_users.json"

def load_users():
if not os.path.exists(USERS_FILE):
return {} try:
    with open(USERS_FILE) as f: return json.load(f)
except: return {}

      def save_users(u):
          with open(USERS_FILE, "w") as f: json.dump(u,


          indent=2) def get_user(email): return


          load_users().get(email)
```

```
def upsert_user(email, name,
picture): users = load_users()
if email not in users:
users[email] = {"name":name,"email":email,"picture":picture,
"xp":0,"quiz_attempts":0,"total_score":0, "topics_studied":[],"quiz_history":[]}
else:
 users[email]["name"]= name users[email]["picture"] = picture save_users(user
s) return
users[email]
t.set_page_config(page_title="COGNIFY — AI Learning",
page_icon="□", layout="wide", initial_sidebar_state="expanded")
CSS = """
<style>
@impo
rt
url('https://fonts.googleapis.com/css2?family=Nunito:wght@400;600;700;800;900
&family=Poppins:wght@400;500;600;700&family=Fredoka+One&display=swap');
:root {
--bg: #f0f4ff;
--white:#ffffff;
--surface:  #ffffff;
--surface2: #f7f9ff;
--border:   #e0e7ff;
--primary:  #4f46e5;
--primary2: #818cf8;
--accent:   #f59e0b;
--accent2:  #10b981;
--pink:#ec4899;
--sky:#0ea5e9;
--text: #1e1b4b;
--muted:  #6b7280;
--danger:   #ef4444;
--radius:   16px;
--radius-lg:28px;
```

```
--shadow-lg:0 8px 40px rgba(79,70,229,.18);
        }
      html, body,
        [data-
        testid="stApp"
        ] {
        background: var(--bg)
        !important;
        font-family: 'Nunito', sans-serif
        !important; color: var(--text)
        !important;
        }
        /* Subtle dot-
        grid background
        */ [data-
        testid="stApp"]
        { background-
        image:
        radial-gradient(circle, #c7d2fe 1px, transparent 1px) !important; background-size: 28px
        28px !important;
        background-color: #f0f4ff !important;
        }
        [data-testid="stSidebar"] {
        background: linear-gradient(180deg, #4f46e5 0%, #6366f1 60%, #818cf8 100%)
        !important; border-right: none !important;
        }
        [data-testid="stSidebar"] * { color: #fff !important; }
        [data-testid="stSidebar"]  .stMetric  {  background:  rgba(255,255,255,.15)
        !important;  border:1px  solid  rgba(255,255,255,.2)  !important;  border-
        radius:12px !important; } [data-testid="stMetricValue"] { color:#fff !important;
        font-family:'Fredoka One'
        !important; font-size:1.6rem !important; }
        [data-testid="stMetricLabel"] { color:rgba(255,255,255,.75) !important;
        font- size:.7 rem !important; text-transform:uppercase
```

```
h1,h2,h3,h4,h5,h6 { font-family:'Fredoka One',sans-serif !important; color:var(--text)
!important; letter-spacing:.01em; }
.stButton > button {
background: linear-gradient(135deg, var(--primary), #6366f1) !important;
color: #fff !important;
border: none
!important; border-
radius: 50px
!important;
font-family: 'Nunito', sans-serif
!important; font-weight: 800
!important;
font-size: .88rem
!important; letter-
spacing: .04em
!important; padding: .6rem
1.6rem !important; transition:
all .25s ease !important;
box-shadow: 0 4px 16px rgba(79,70,229,.35)
!important; text-transform: uppercase !important;
}
.stButton > button:hover {
transform: translateY(-3px) scale(1.02) !important;
box-shadow: 0 8px 28px rgba(79,70,229,.5) !important;
}
.stButton > button:active { transform: translateY(0) !important; }
.stButton > button * { color:#fff !important; }

.stTextInput input, .stTextArea
textarea { background: var(--
white)
!important; border: 2px solid
var(-- border) !important;
border-radius:
```

```
        var(-- text) !important;
  font-family: 'Nunito', sans-serif
  !important; font-size: .95rem
  !important;
  }
.stTextInput input:focus, .stTextArea
  textarea:focus { border-color: var(--
  primary)
  !important;
  box-shadow: 0 0 0 4px rgba(79,70,229,.12) !important;
  }
  .stTextInputlabel .stTextArealabel
  !important; font-size:.82rem !important; }

  div[data-baseweb="tab-
  list"] { background: var(--
  white)
  !important; border-radius:
  50px !important; padding: 5px
  !important;
  gap: 4px !important;
  border: 2px solid var(--border)
  !important; box-shadow: var(--
  shadow) !important;
  }
  logged_in":False,"user":None,"role":None}
  if "login_step" not in st.session_state: st.session_state.login_step = "role"
  if "oauth_token_consumed" not in st.session_state: st.session_state.oauth
  _token_consumed =
  False
  if result and result.get("token"):
  access_token =
  result["token"].get("access_token", "") if
  access_token:
```

```
# Mark token as consumed BEFORE any st.rerun()
st.session_state.oauth_token_consumed = True try:
ui = requests.get(
"https://www.googleapis.com/oauth2/v3/userinfo",
headers={"Authorization": f"Bearer {access_token}"}, timeout=10,
).json()
email = ui.get("email", "") name =
ui.get("name", email) picture =
ui.get("picture", "") if not email:
st.error("Could not retrieve email. Please try again.")
st.session_state.oauth_token_consumed = False
else:
upsert_user(email, name, picture)
# Fully reset login state and mark logged in
st.session_state.auth = {"logged_in": True, "user": email, "role": "user"}
st.session_state.login_step = "role" st.session_state.oauth_token_consumed = False
st.rerun()
except Exception as e:
st.session_state.oauth_token_consumed = False
filtered = [(e,u) for e,u in students if not search
or search.lower() in u.get("name","").lower()
or search.lower() in e.lower()]
def get_client():
api_key = os.environ.get("API_KEY")
if not api_key: st.error("Missing API_KEY environment variable.");
st.stop() return genai.Client(api_key=api_key)

client = get_client()
WIKI_H = {"User-Agent":"Mozilla/5.0"}; GFG_H =
{"User-Agent":"Mozilla/5.0"};
W3_H =
{"User-
Agent":"Mozilla/5.0"
} def _trunc(text,
```

```
limit=800):
if not text: return
"" s="
".join(str(text).spli
t()) if
len(s)<=limit:
return s
cut=s[:limit] for
sep in[".", "!", "?"]:
i=cut.rfind(sep)
    if i!=-1 and i>=int(limit*.6):
return cut[:i+1] si=cut.rfind(" ")
        return (cut[:si]+"...") if si!=-1 else
cut+"..." def _gfg_urls(q, n=3):
if not DDGS_AVAILABLE:
 return [] urls=[]
 try:
 with DDGS() as d:
 for r in d.text(f"site:geeksforgeeks.org {q}",
 max_results=8): link=r.get("href") or r.get("link") or
 r.get("url")
 if link and "geeksforgeeks.org" in str(link) and link not in urls:
 urls.append(link)
            if len(urls)>=n:
 break except: pass
 return urls
 def _w3_urls(q, n=3):
 if not DDGS_AVAILABLE:
 return [] urls=[]
 try:
 with DDGS() as d:
 for r in d.text(f"site:w3schools.com {q}", max_results=8):
 link=r.get("href") or r.get("link") or r.get("url")
 if link and "w3schools.com" in str(link) and link not in urls:
```

```
urls.append(link)
if len(urls)>=n: break
except: pass
return urls
def wiki_summary(topic):
for v in [topic.strip(), topic.strip().replace(" ","_"),
topic.strip().title()]: try:
r=requests.get(f"https://en.wikipedia.org/api/rest_v1/page/summary/{urllib.parse.quo
t e(v)}", timeout=10, headers=WIKI_H)
if r.status_code==200:
ex=r.json().get("extract","")
        if ex and len(ex)>40: return
ex.strip()
except: pass
  return
None
def
get_summary(to
p ic):
s=wiki_summary
(topic)
if s and len(s)>30: return
_trunc(s,1200) if
DDGS_AVAILABLE:
try:
with DDGS() as d:
for r in d.answers(topic):
t=r.get("text") or r.get("abstract") if isinstance(r,dict) else None if
t and len(t)>30: return _trunc(t,1200)
    exce
pt: pass
return
None
def _pil(img):
```

```
if img is None: return None
    return img.convert("RGB") if img.mode
in("RGBA","P") else img def _hash(img):
try: return
hash(img.resize((64,64),Image.Resampling.LANCZOS).convert("L").tobytes()) except:
return hash(img.tobytes())
def _bad_url(url):
if not url: return True
in("almanac","manuscript","archive.org","antique","newspap
e r"))
def
_dedup(i
mgs):
seen,out
= set(),[]
for img
in imgs:
if img is None: continue
h=_hash(img)
    if h not in seen:
seen.add(h);
out.append(img) return
out def
_placeholder(topic):
w,h=400,260
# bright placeholder
img=Image.new("RGB",(w,h),color=(238,242,2
55)) draw=ImageDraw.Draw(img)
draw.rectangle([(3,3),(w-4,h-
4)],outline="#4f46e5",width=3)
title=(str(topic)[:28]+"…") if len(str(topic))>28
else
str(topic) draw.text((w//2-55,h//2-
15),title,fill="#1e1b4b") draw.text((w//2-
```

```
38,h//2+12),"Visualization",fill="#6b7280") return
img
def_extract_urls(
html):
    urls=set()
urls.update(re.findall(r'src=["\'](https?://[^"\']+)["\']',html,re.I)
)
urls.update(re.findall(r'data-
src=["\'](https?://[^"\']+)["\']',html,re.I)) for s in
re.findall(r'srcset=["\']([^"\']+)["\']',html,re.I):
for p in s.split(","):
u=p.strip().split(" ")[0]
        if
u.startswith("http"):
urls.add(u)          return
list(urls) def
_dl(urls,headers,
n=5):
    out,seen=[],
set() for url in
urls:
low=url.low
er()
        if any(x in low for x
in["logo","icon","avatar","favicon","pixel","banner","ads","spacer"]):
continue try:
r=requests.get(url,headers=headers,timeout=10)
if r.status_code!=200 or len(r.content)<800: continue
img=_pil(Image.open(BytesIO(r.content)))
if not img or img.size[0]<220 or img.size[1]<180: continue
h=_hash(img)
if h in seen: continue
seen.add(h);
out.append(img) if
```

```
len(out)>=n: break
except:
continue
return
out
CSE_KEY="AIzaSyBHY0WmJs8VcjmsGRdoxWP5yQuxE
cs Arc8"; CSE_CX="865d79e8bd12c4859"
def
_cse_urls(q,
n=5): try:
    r=requests.get("https://www.googleapis.com/customsearch/v1",
params={"key":CSE_KEY,"cx":CSE_CX,"q":q,"searchType":"image","num":n,"
safe":"active"}
,timeout=10)
if r.status_code!=200: return []
return [i["link"] for i in r.json().get("items",[]) if
i.get("link","").startswith("http")][:n] except: return []

def get_cse_images(topic,n=5):
if not(topic or"").strip():
return [] imgs=[]
for q in[f"{topic} diagram",f"{topic}
architecture",f"{topic} workflow"]: if len(imgs)>=n:
break imgs.extend(_dl(_cse_urls(q,n-
len(imgs)),WIKI_H,n-
len(imgs))) return _dedup(imgs)[:n]

def gen_img(prompt):
t=prompt.split("-
")[0].strip()
imgs=fetch_images(t)
return imgs[0] if imgs
else
_placeholder(t) with col_in:
st.markdown('<div class="card">',
```

```
unsafe_allow_html=True) st.markdown("""
<div style="font-family:'Fredok One';font-
size:1.2rem;color:#4f46e5; margin-bottom:.9rem;">
</div
unsafe_allow_html=True)
if MIC_AVAILABLE:
tc,mc =
st.columns([3,1])
with mc:
st.markdown('<divstyle="font-size:.7rem;color:#9ca3af;margin-
top:1.5rem;text- align:center;">🎙 Speak</div>', unsafe_allow_html=True)
vt =speech_to_text(language="en",use_container_width=True,just_once=True,
key="voice_m")
if vt:
st.session_state["ti_m"]=
vt with tc:
        input_text = st.text_area("Topic", placeholder="e.g. Photosynthesis,
Arrays, Climate Change…", height=115, key="ti_m",
label_visibility="collapsed") else:
input_text = st.text_area("Topic", placeholder="e.g. Photosynthesis,
Arrays, Climate Change…", height=115, label_visibility="collapsed")
with col_viz:
if st.session_state.vault:
aidx =
st.session_state.active_idx data
= st.session_state.vault[aidx]
analysis = data["analysis"]
tab1,tab2,tab3,tab4 = st.tabs(["🖼 Gallery","🗺 Mind Map","📈 Flow","📝 Quiz"]) else:
st.markdown("""
<div style="display:flex;flex-direction:column;align-items:center;justify-
content:center; height:420px;text-align:center;">
<div style="font-size:5rem;margin-bottom:1rem;">
<span class="float">□</span>
</div>
```

7. OUTPUT SCREENS

Step 1: System Entry – Landing Page

After running the Streamlit application, a localhost webpage opens showing the Cognify login interface.

The user selects a role Student or Admin from the role selection screen.

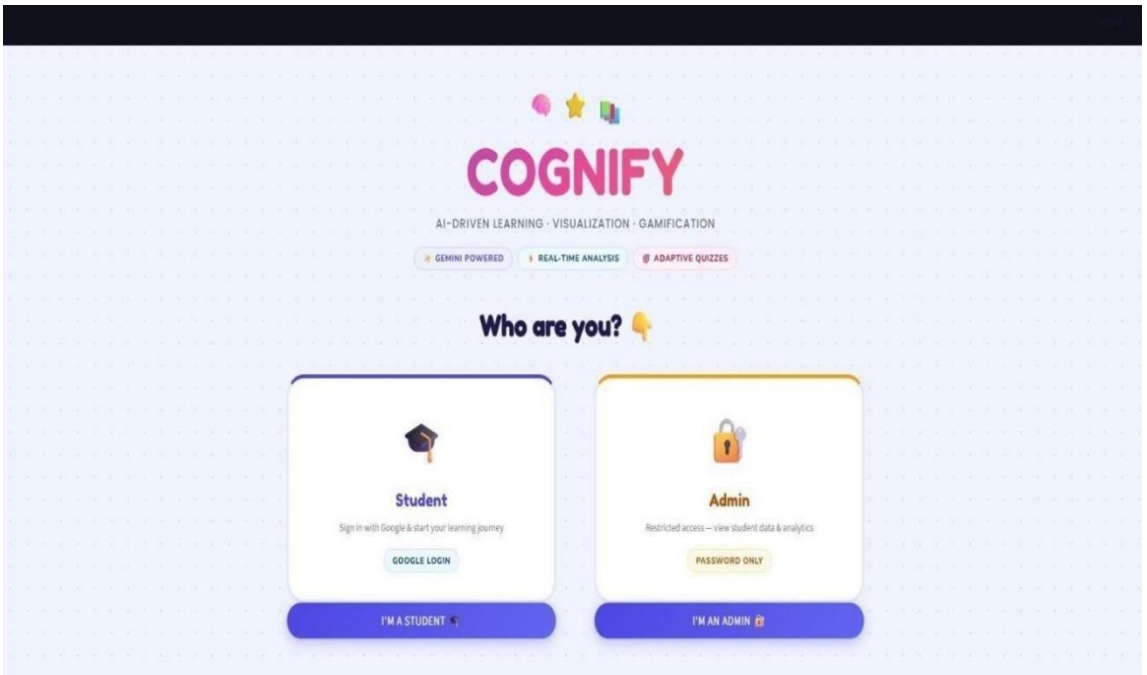

Fig 7.1: User Interface

Step 2: Student Authentication using Google OAuth

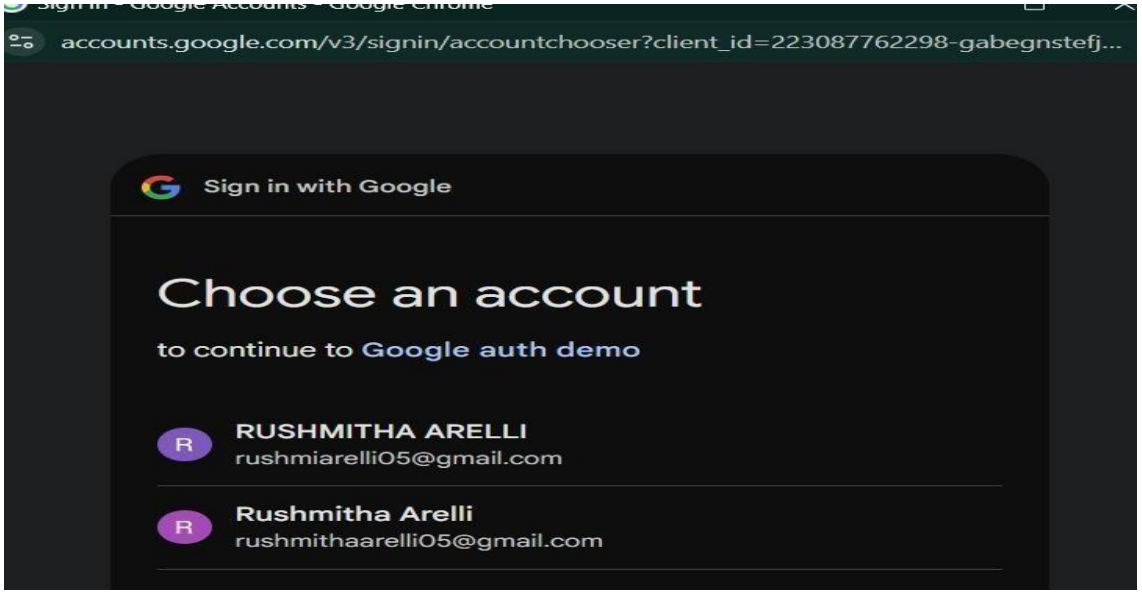

Fig 7.2: Student Authentication

Step 3: Admin Authentication

After successful login, the administrator can access the Admin Dashboard, which provides an overview of the learning platform.

Fig 7.3: Admin Authentication

Step 4: Admin Dashboard

After successful login, the administrator can access the Admin Dashboard, which provides an overview of the learning platform.

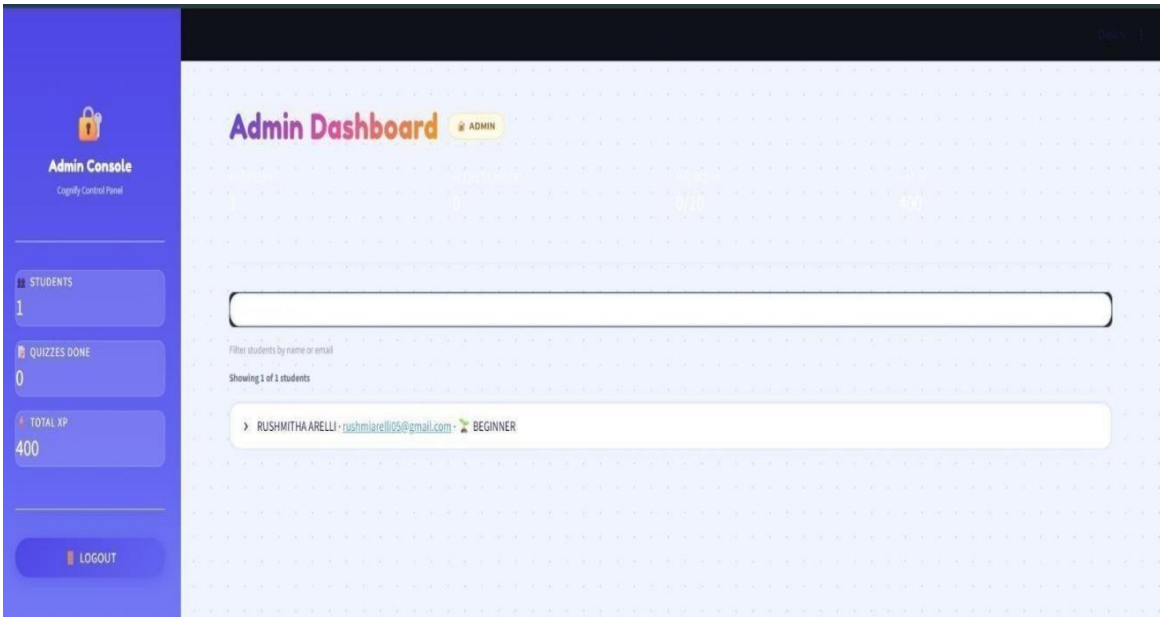

Fig 7.4: Admin Dashboard

Step 5: Student learning Dashboard

Once the student logs in successfully, they are redirected to the Student Learning Dashboard.

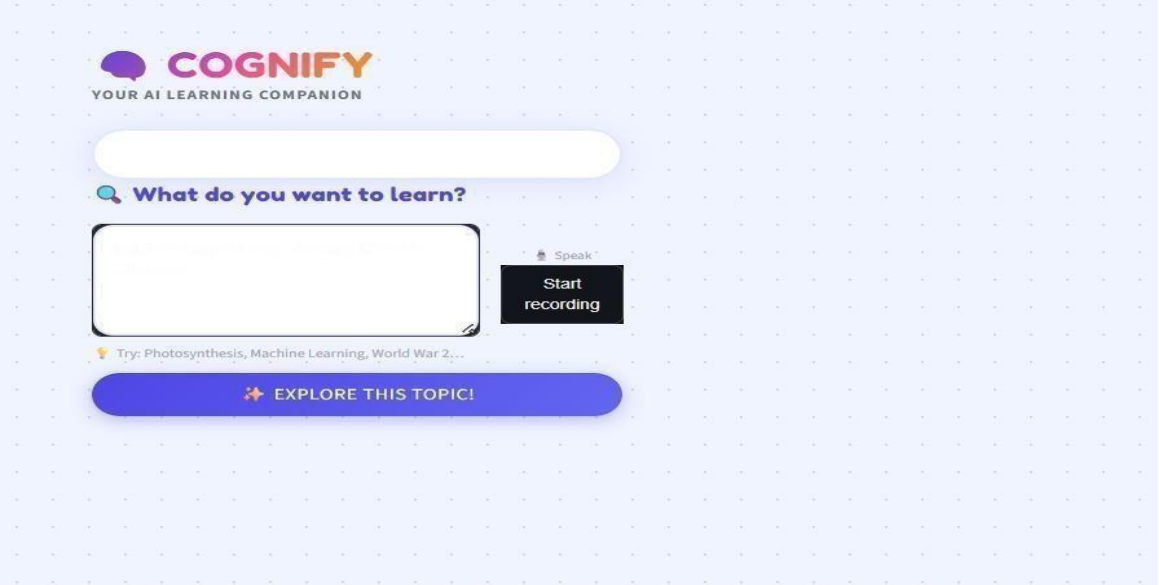

Fig 7.5: Student learning Dashboard

Step 6: Topic Input for AI Learning

The platform provides a text input box where students can enter any topic they want to learn.

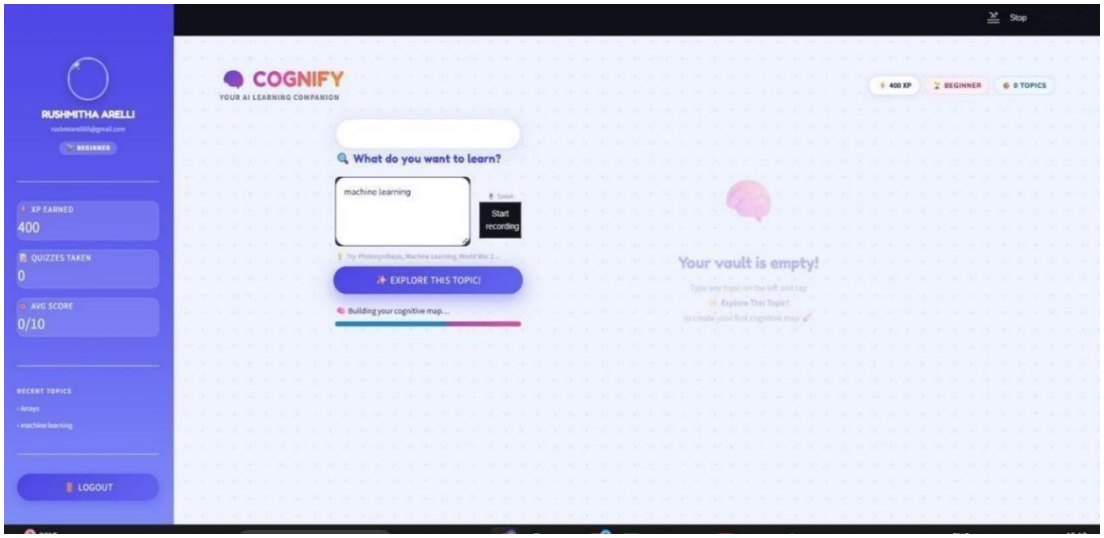

Fig 7.6: Topic Input

Step 7: AI-Based Content Generation

Once the topic is submitted, the system uses AI to generate educational content.

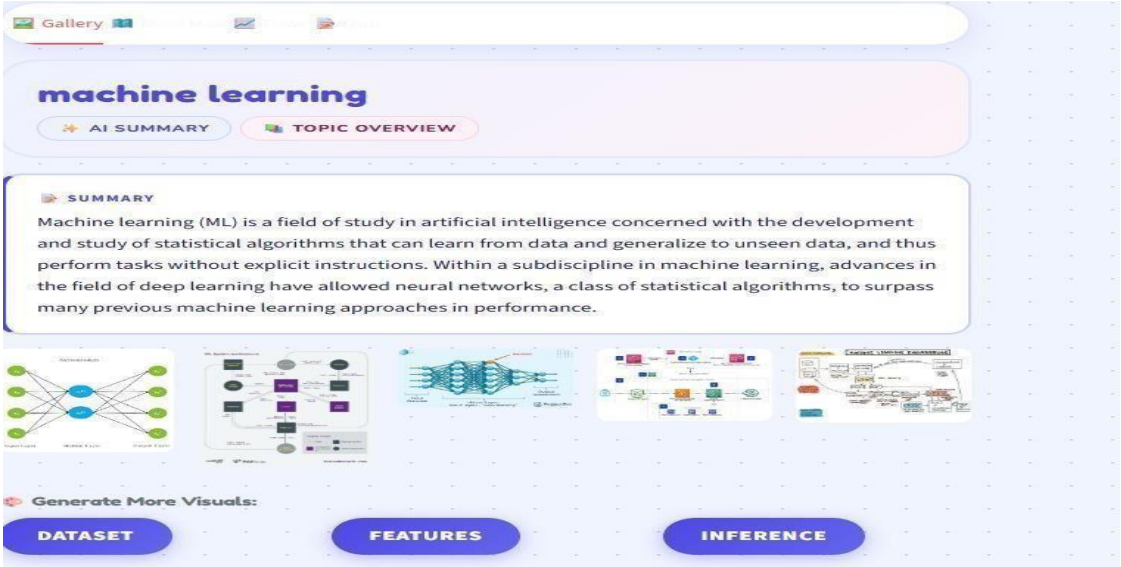

Fig 7.7: AI-Based Content Generation

Step 8: Concept Mind Map Generation

COGNIFY automatically generates a concept mind map for the selected topic.

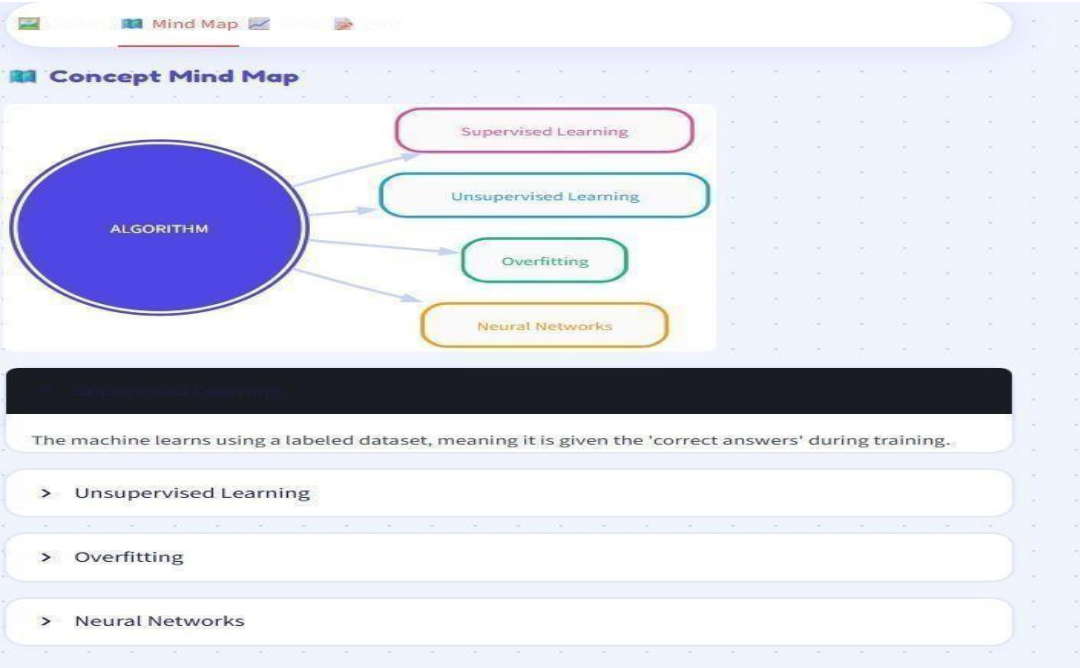

Fig 7.8: Concept Mind Map Generation

Step 9: Learning Flow Visualization

The platform also generates a step-by-step learning process flow for the selected topic.

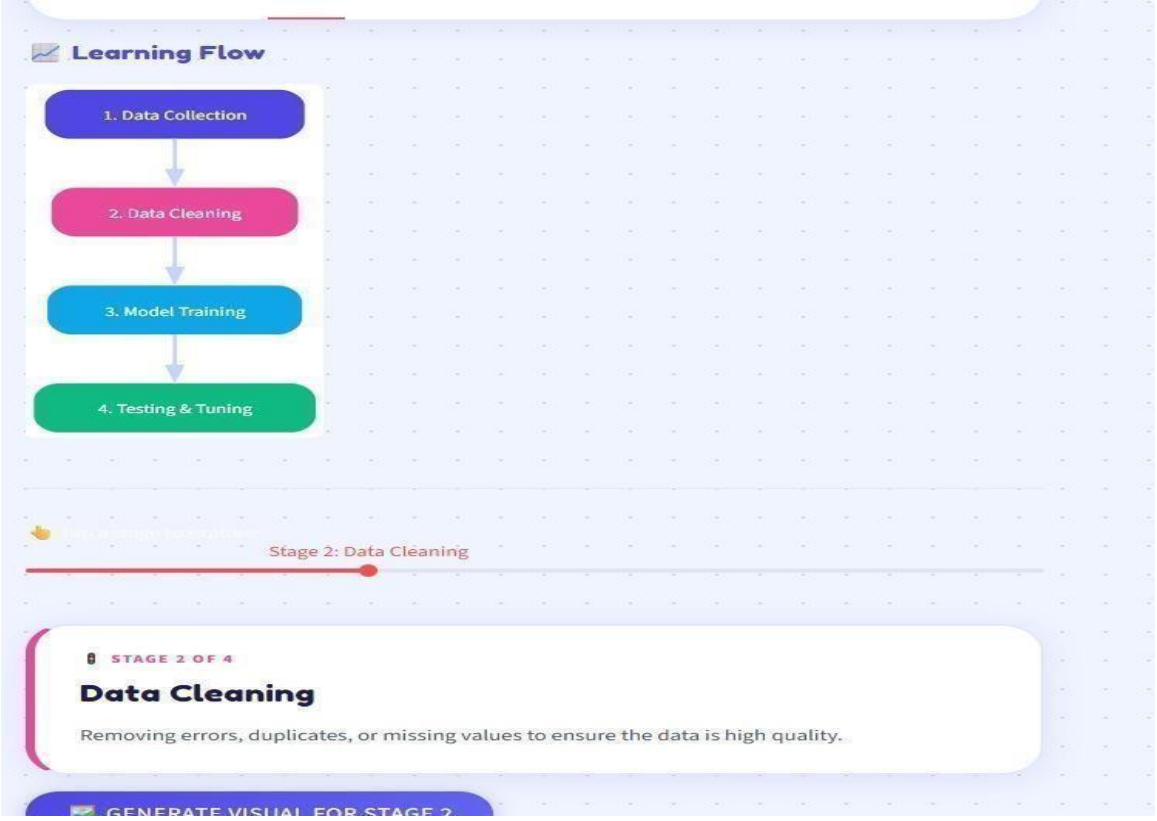

Fig 7.9: Learning Flow Visualization

Step 10: AI Reinforced Knowledge Quiz

After learning the topic, the system generates an automated knowledge quiz.

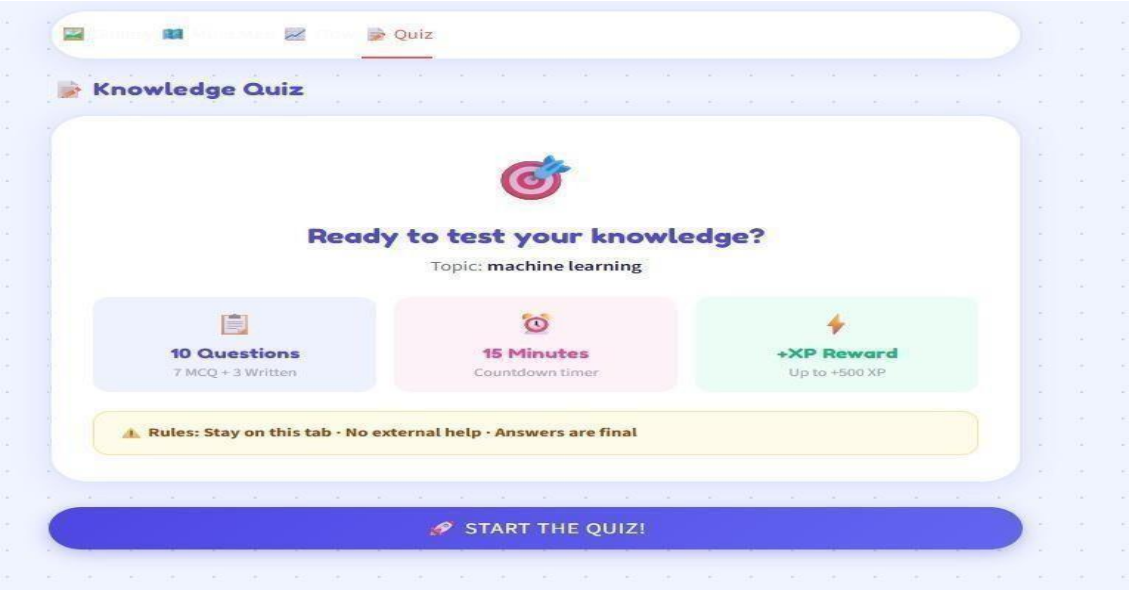

Fig 7.10: AI Reinforced Knowledge Quiz

Step 11: Tab Switching Detection (Anti-Cheating Mechanism)

To maintain exam integrity, the system monitors user activity during the quiz.

If the student switches browser tabs, the system triggers a warning message.

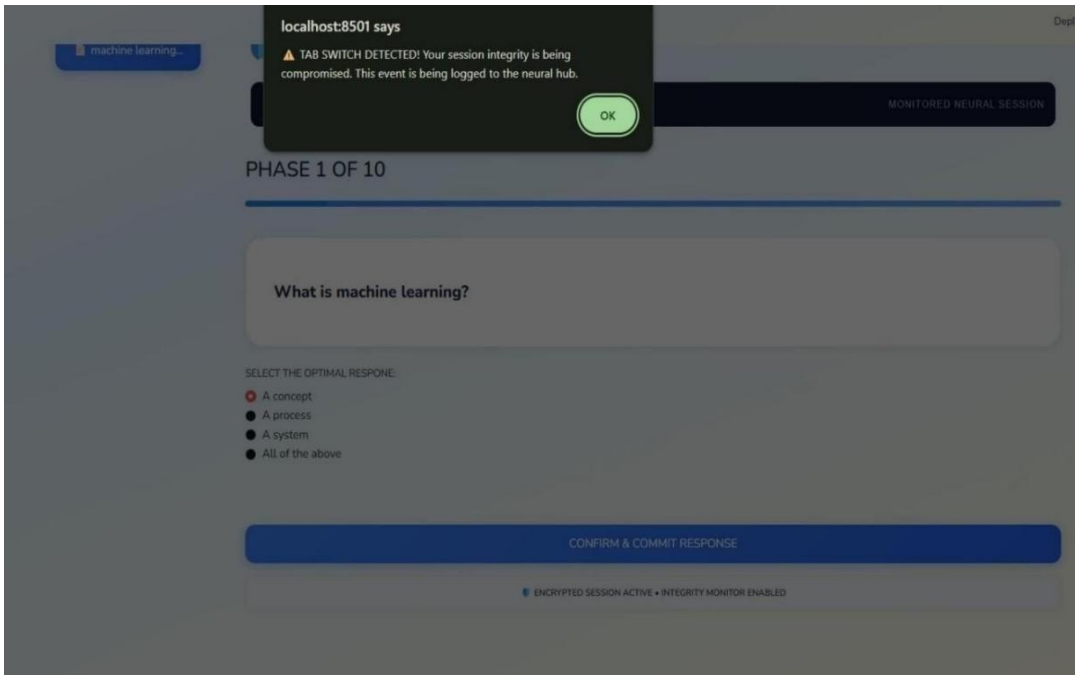

Fig 7.11: Tab Switching Detection

Step 12: Answer Submission and Confirmation

Students select their answers and submit them using the Confirm & Commit Response button.

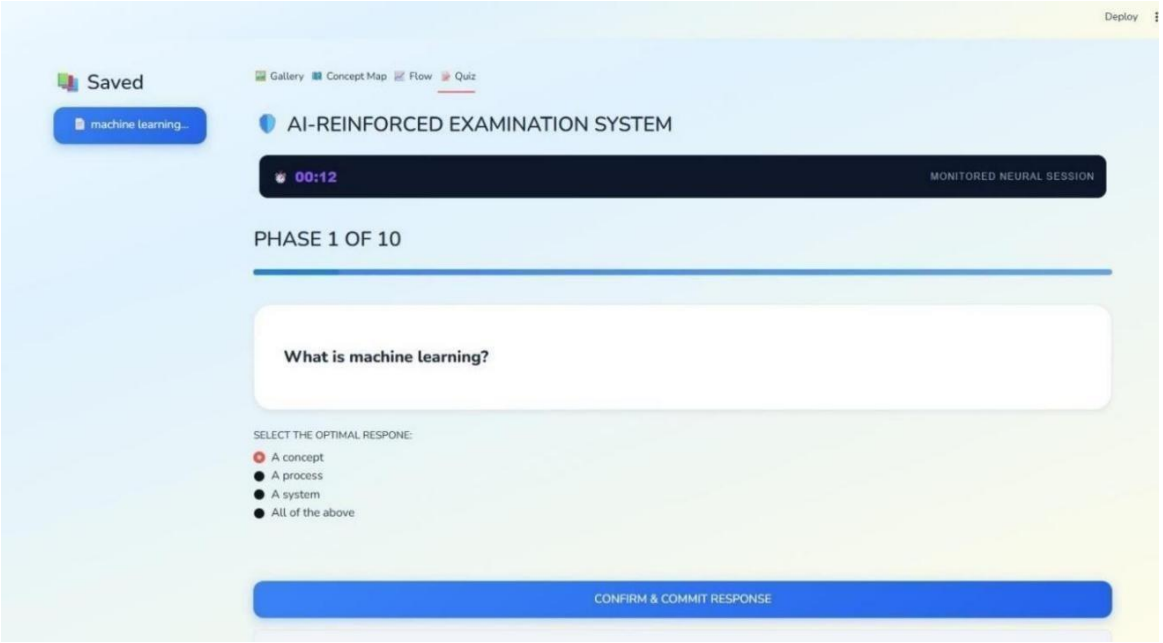

Fig 7.12: Answer Submission and Confirmation

Step 13: Quiz Result Evaluation

After completing the quiz, the system evaluates the responses automatically.

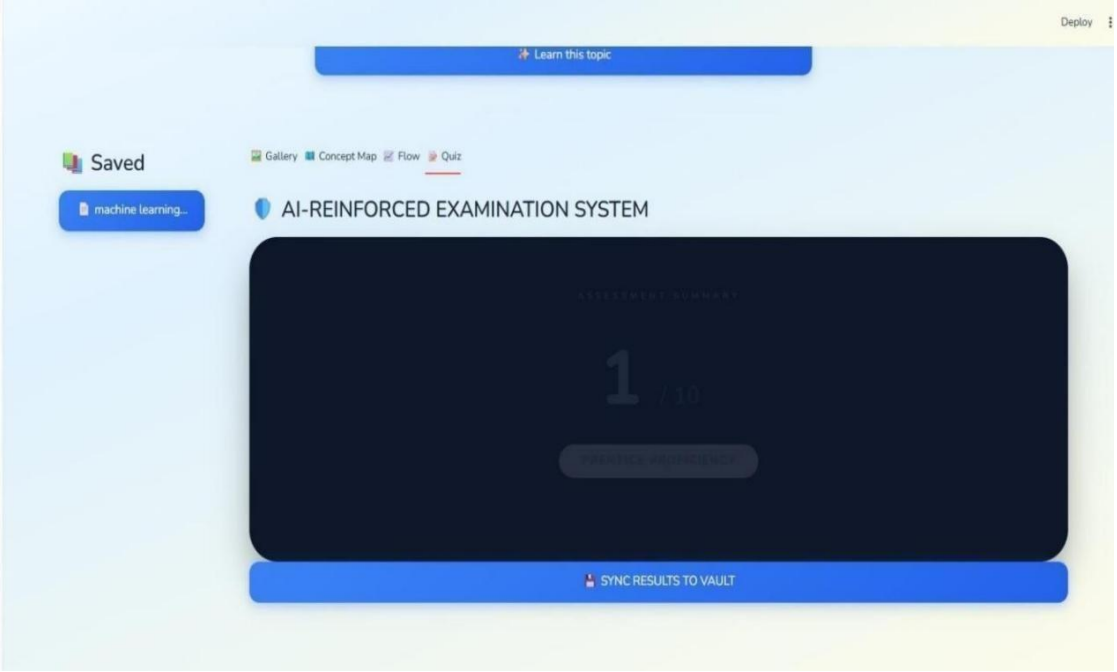

Fig 7.13: Quiz Result Evaluation

Step 14: Student Performance Profile

Each student has a performance profile that tracks learning activity.

Fig 7.14: Student Performance Profile

8. CONCLUSION

The Cognify platform provides a modern solution to the limitations of traditional learning methods by transforming complex academic content into clear, interactive, and visually rich formats. Through the use of Artificial Intelligence and Natural Language Processing, the system converts text and voice-based queries into meaningful visuals such as flowcharts, mind maps, step-by-step diagrams, and 3D models. This visual-first approach not only simplifies understanding but also supports faster comprehension, better retention, and deeper engagement. By integrating personalized explanations and adaptive responses, Cognify ensures that each user receives learning support tailored to their pace and style.

In addition to visual learning, Cognify enhances overall motivation and consistency through built-in gamification features like quizzes, points, and badges, promoting a more enjoyable learning experience. The platform also tracks user progress, enabling continuous improvement and self-learning. Overall, Cognify bridges the gap between complex educational content and user understanding, offering a unified, intelligent, and highly interactive learning environment. It serves as a powerful step toward future-ready education, making learning more accessible, personalized, and impactful for all users.

FUTURE SCOPE

Integration of Multi-Modal Learning Content:

Future improvements of the COGNIFY platform can include the integration of multiple forms of educational content such as videos, animations, interactive diagrams, and audio explanations. Combining these different learning formats will help students understand complex concepts more effectively and cater to different learning styles. This multi-modal learning approach can significantly improve engagement and knowledge retention.

Advanced AI-Based Learning Assistance:

The platform can be enhanced with more advanced AI models capable of providing deeper explanations, answering follow-up questions, and generating more accurate learning materials. Future versions could implement conversational AI tutors that guide students step-by- step through difficult topics, helping them understand concepts more clearly and interactively.

Personalized Learning Recommendations:

Future versions of the system can incorporate adaptive learning mechanisms that analyze student performance, quiz results, and learning patterns. Based on this analysis, the platform can recommend personalized topics, adjust quiz difficulty levels, and provide targeted learning resources to help students focus on their weak areas and improve their understanding.

Enhanced Visualization and Interactive Simulations:

The project can be further improved by adding interactive visualizations such as animated concept maps, simulations, and 3D diagrams. These tools will help students visualize abstract concepts and processes more effectively, making learning more intuitive and engaging.

Voice-Enabled Learning Interface:

Future development may include a voice-enabled AI assistant that allows students to interact with the platform through voice commands. Students could ask questions verbally and receive spoken explanations, which would make the learning experience more natural and accessible.

REFERENCES

- Teacher Perspectives On AI-Driven Gamification: Impact On Student Motivation, Engagement, And Learning Outcomes

  https://www.proquest.com/openview/5d6449c33d46617b6947037a00c8fbc7/1?pq-origsite=gscholar&cbl=6515896

  Authors: A Alenezi - Information Technologies and Learning Tools

- AI-Driven Personalized Learning Paths: Enhancing Student Engagement Through Gamification and Interdisciplinary Integration

  https://ieeexplore.ieee.org/abstract/document/11158040/authors#authors

  Authors: A Adiyono, AP Rahayu, A Besari

- Gamification and Artificial Intelligence in Education: A Review of Innovative Approaches to Fostering Critical Thinking

  https://ieeexplore.ieee.org/abstract/document/11021565

  Author: AM KassenKhan, AN Moldagulova, VV Serbin

- AI-Powered Gamification and Interactive Learning Tools for Enhancing Student Engagement

  https://www.igi-global.com/chapter/ai-powered-gamification-and-interactive-learning-tools-for- enhancing-student-engagement/370084

  Authors: BG Banik, A Gullapelly Education Through AI and Data Science

- Dynamic Horizon:AI and Gamification for Reshaping Higher Education

  https://www.taylorfrancis.com/chapters/edit/10.1201/9781003469315-14/dynamic-horizon- srimathi- anitha

  Authors: S.Srimathi, D. Anitha